

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Chương 7 Thiết kế Hệ thống

Thiết kế - Kiến trúc

Architecture Design

hung.letan@hust.edu.vn

0913030731

ONE LOVE. ONE FUTURE.

Nội dung

- Khái niệm ban đầu về Thiết kế HTTT
  - Phân biệt được sự khác nhau giữa Phân tích và Thiết kế.
- Cung cấp kiến thức cần thiết để hiểu về kiến trúc hệ thống
  - Khái niệm kien trúc trong truyền đạt ý tưởng thiết kế Hệ thống phần mềm
  - Mẫu thiết kế kiến trúc phần mềm."
- Chi tiết:
  - Hiểu rõ định nghĩa và vai trò của Kiến trúc Hệ thống, kiến trúc phần mềm
  - Hiểu giống và khác giữa xu hướng kiến trúc phổ biến.
  - Biết cách sử dụng các sơ đồ C4, UML trực quan hóa thiết kế kiến trúc hệ thống:
    - Component Diagram,
    - Deployment Diagram,
    - Package Diagram

Thiết kế (Design) hệ thống

- (System Design)
  - là giai đoạn của vòng đời phát triển hệ thống (SDLC) nhằm quyết định cách thức xây dựng một hệ thống thông tin (how to build the system).
- Quá trình này được xem như một sự tiến hóa
  - Trong OOAD, ranh giới giữa phân tích và thiết kế thường không rõ ràng.
  - các mô hình phân tích được tinh chỉnh và bổ sung các chi tiết kỹ thuật để trở thành mô hình thiết kế
- Giai đoạn thiết kế này các mô hình logic được phát triển thành các mô hình vật lý

Sự chuyển tiếp từ Phân tích sang Thiết kế với UML

- Mô hình Logic (Logical Model):
  - Được tạo ra trong giai đoạn phân tích,
  - mô tả những gì hệ thống phải làm,
  - độc lập với công nghệ triển khai.
  - Các mô hình này gồm
    1. sơ đồ lớp miền vấn đề (domain model class model),
    2. sơ đồ use case (use case diagrams), và
    3. sơ đồ tuần tự(system sequence diagrams).
- Mô hình Vật lý (Physical Model):
  - Được tạo ra trong giai đoạn thiết kế,
  - mô tả cách thức hệ thống sẽ được xây dựng.
  - Mô hình này bổ sung các chi tiết kỹ thuật vào mô hình logic như
    - Class/Object xác định các lớp phần mềm cụ thể,
    - Method/ các phương thức, chữ ký phương thức (method signatures),
      - cách các đối tượng sẽ tương tác trong môi trường triển khai cụ thể.
    - Các sơ đồ khác ….
      - Component Diagram, Deployment Diagram, và Package Diagram

Tại sao cần làm rõ thành PhaThiết kế?

- Phân tích hệ thống tập trung vào việc xác định và định nghĩa vấn đề.
- Thiết kế hệ thống tập trung vào việc giải quyết vấn đề và tìm ra giải pháp.

- Thiết kế là cây cầu nối:
  - Nó chuyển đổi các yêu cầu của người dùng (từ pha phân tích) thành một bản thiết kế chi tiết (blueprint) cho các nhà phát triển, lập trình viên.
- Thiết kế là Nền tảng cho chất lượng:
  - Một thiết kế tốt sẽ tạo ra một hệ thống hiệu quả, đáng tin cậy và dễ bảo trì
  - Thiết kế tốt sẽ che giấu đi sự phức tạp bên trong hệ thống.
- Tránh "Analysis Paralysis": (liệt)
  - Thiết kế giúp chuyển từ giai đoạn suy nghĩ sang hành động,
  - tránh tình trạng phân tích quá mức mà không có tiến triển.

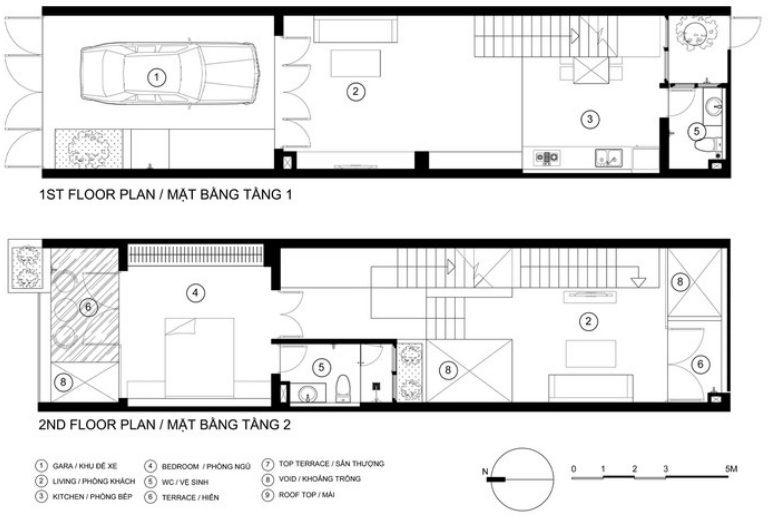
4 lĩnh vực chính của thiết kế trong kỹ thuật phần mềm..

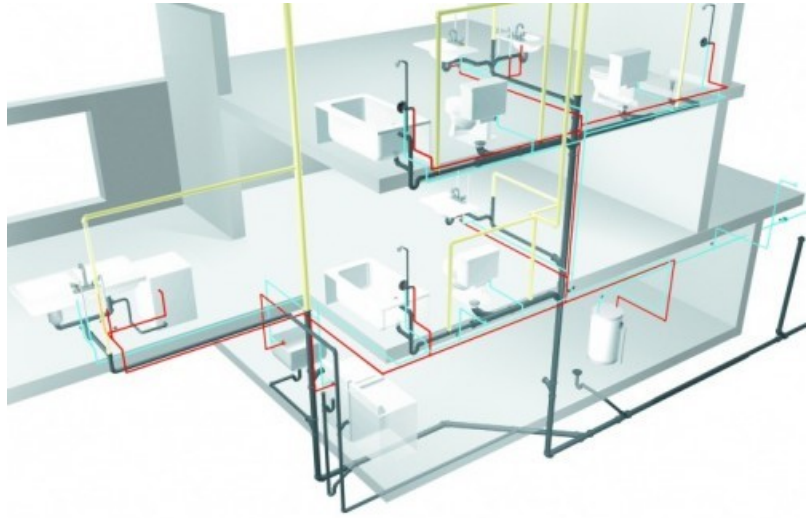
- Giai đoạn thiết kế bao gồm nhiều hoạt động nhằm tạo ra một bản thiết kế chi tiết (blueprint) cho hệ thống, gồm
  - 1. Thiết kế Kiến trúc (Architectural Design):
    - Thiết kế cấp cao xác định cấu trúc tổng thể của hệ thống, gồm
      - các thành phần chính (mô-đun),
      - chức năng của chúng và
      - cách chúng tương tác với nhau.
  - 2. Thiết kế Lớp và Phương thức (Class and Method Design):
    - Tinh chỉnh Sơ đồ Lớp
    - Thiết kế Tương tác Đối tượng.
  - 3. Thiết kế Dữ liệu (Data Design):
    - Thiết kế cách thức lưu trữ dữ liệu lâu dài (persistent),
    - chi tiết cách dữ liệu được lưu trữ, quản lý và truy cập trong hệ thống
  - 4. Thiết kế Giao diện Người dùng (User Interface Design):
    - Quyết định cách người dùng sẽ tương tác với hệ thống,

Khái niệm kiến trúc

- Architecture - Kiến trúc XD ?
- System architecture – (IS)
  - Ở giai đoạn Phân tích
    - trả lời câu hỏi 'Hệ thống cần làm gì?'.
  - Giai đoạn Thiết kế
    - trả lời câu hỏi 'Hệ thống sẽ được xây dựng như thế nào?'.

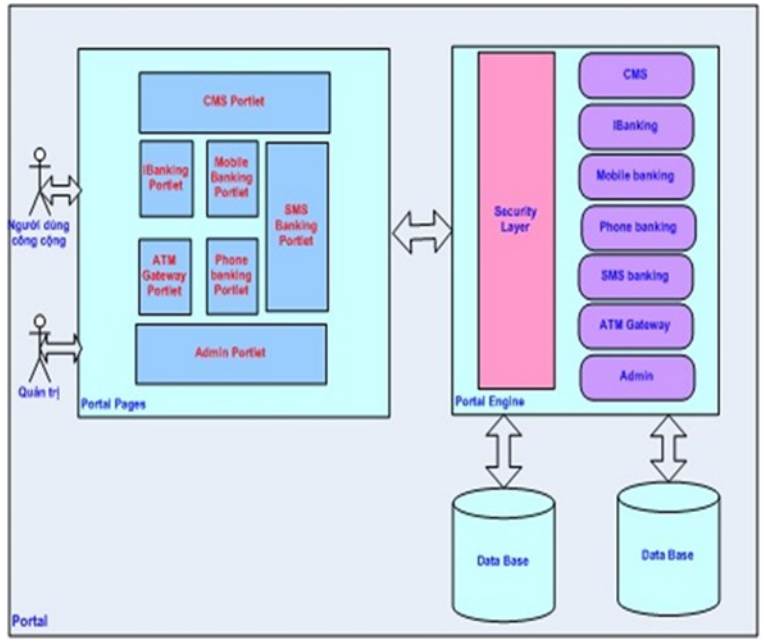






System architecture Kiến trúc hệ thống

- System architecture
  - không phải là các thành phần riêng lẻ, mà là cách chúng được tổ chức và liên kết với nhau để tạo thành thể thống nhất.
- Tổ chức cơ bản gồm:
  1. Các thành phần (components) của hệ thống.
  2. Mối quan hệ: giữa các thành phần và với môi trường bên ngoài.
  3. hành vi: cách chúng tương tác với nhau
- Kiến trúc hệ thống được xây dựng dựa trên
  - logic nghiệp vụ và nhu cầu hoạt động thực tế của tổ chức.
- Hệ thống được thiết kế dựa trên quan điểm về tầm nhìn.
  - hướng tới một bức tranh tổng thể, mục tiêu dài hạn
  - đáp ứng được tầm nhìn kinh doanh,
  - thường do Kiến trúc sư phần mềm dẫn dắt

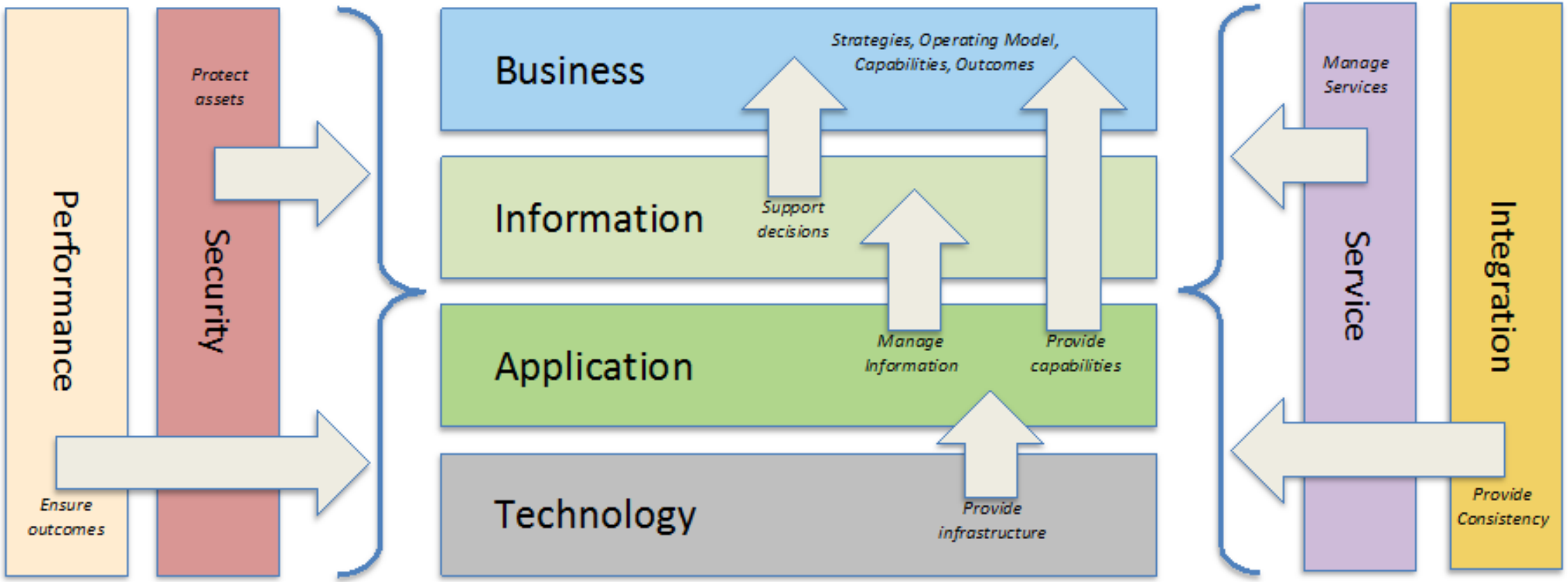


Enterprise architecture – Kiến trúc Doanh nghiệp

- Enterprise architecture
  - là khuôn khổ lập kế hoạch chiến lược liên kết CNTT của toàn bộ tổ chức với các mục tiêu kinh doanh của tổ chức,
  - EA đảm bảo rằng mọi bộ phận, quy trình và khoản đầu tư công nghệ đều hoạt động hài hòa để đạt được các mục tiêu dài hạn
- EA đang nhanh chóng trở thành công cụ chính để cải thiện sự liên kết CNTT & kinh doanh.“
- 1992 -Enterprise Architecture Planning (EAP)” Stephen Spewack - -
  - thiết lập phương pháp tiếp cận theo hướng từ trên xuống, thúc đẩy đối với quy hoạch kiến trúc, trở thành yếu tố nền tảng cho nhiều khuôn khổ EA sau này, TOGAF, FEAF

Thành phần của Enterprise Architecture

- Hầu hết các khung kiến trúc hiện đại đều tổ chức kiến trúc doanh nghiệp thành bốn lớp chính, thường được gọi là ngăn xếp BDAT
  - Business Architecture.
    - chiến lược kinh doanh, quản trị và các quy trình chính.
    - Nó phác thảo cách thức tổ chức tạo ra giá trị
  - Data Architecture:
    - Tổ chức các tài sản dữ liệu logic và vật lý cũng như các nguồn lực quản lý để đảm bảo thông tin chính xác và dễ truy cập
  - Applications Architecture
    - Cung cấp bản thiết kế cho các hệ thống phần mềm riêng lẻ, sự tương tác giữa chúng và cách chúng hỗ trợ các quy trình kinh doanh
  - Technology Architecture
    - Mô tả cơ sở hạ tầng phần cứng, mạng và phần mềm cần thiết để vận hành các ứng dụng.
- Một số mô hình có thể bổ sung
  - Security, Performance ,tích hợp, dịch vụ trên các miền cốt lõi để giải quyết các mối quan tâm một cách toàn diện.



Enablement, Architecture Program and Team, Organization, Governance

Các khuôn khổ chính

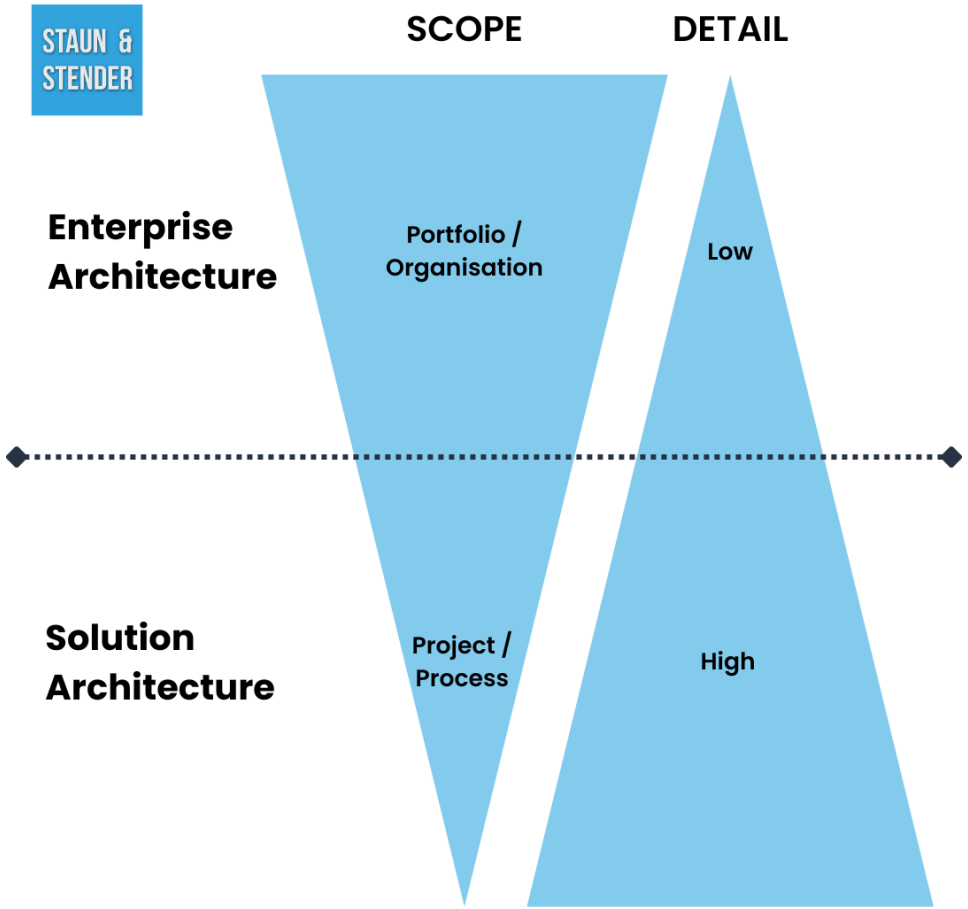
- Các khuôn khổ cung cấp các phương pháp luận tiêu chuẩn hóa và ngôn ngữ chung để triển khai EA.
  - TOGAF (The Open Group Architecture Framework) :
    - Khung kiến trúc phổ biến nhất trên toàn cầu, nổi tiếng với Phương pháp Phát triển Kiến trúc (ADM) để thiết kế và quản lý kiến trúc hệ thống.
  - Khung Zachman :
    - sử dụng lưới 6x6 để xem xét một tổ chức từ các góc độ khác nhau
  - FEAF (Khung kiến trúc doanh nghiệp liên bang) :
    - Ban đầu được thiết kế cho chính phủ Hoa Kỳ để tích hợp các cơ quan liên bang, nhưng cũng được các thực thể tư nhân sử dụng.
  - Phương pháp luận của Gartner:
    - Một cách tiếp cận tập trung vào thực tiễn, nhấn mạnh vào kết quả kinh doanh và sự linh hoạt hơn là các tài liệu cứng nhắc.

Kiến trúc Doanh nghiệp Vs Hệ thống CNTT

- (EA) trung vào bức tranh tổng thể—
  - điều chỉnh chiến lược kinh doanh với CNTT trên toàn bộ tổ chức,
  - thiết lập các tiêu chuẩn và tối ưu hóa toàn bộ hệ thống, giống quy hoạch đô thị;
- Kiến trúc phần mềm (hoặc Kiến trúc giải pháp)
  - tập trung vào thiết kế cụ thể, các thành phần và chi tiết triển khai của từng hệ thống hoặc cơ sở hạ tầng để đáp ứng.
  - xác định cấu trúc nội bộ, mô tả chi tiết bản thiết kế kỹ thuật để xây dựng và tích hợp các phần đó.
  - giống như thiết kế một tòa nhà riêng lẻ trong thành phố đó.
- EA khác gì kiến trúc hệ thống thông tin (phần mềm) ?
  - (EA) là "cái gì" và "tại sao" cho toàn bộ doanh nghiệp, định hướng chiến lược, t
  - (IT System Architecture) là "cách thức" cho từng phần cụ thể

Mối quan hệ hợp tác

- Kiến trúc doanh nghiệp:
  - Phối hợp các giám đốc điều hành cấp cao (CIO, CTO), lãnh đạo các đơn vị kinh doanh và ban quản lý cấp cao để xác định trạng thái "mong muốn" của tổ chức.
  - khung chuyên môn -TOGAF cung cấp các hướng dẫn tiêu chuẩn cho các tổ chức
- Kiến trúc hệ thống CNTT:
  - Phối hợp với các nhóm phát triển, chủ sở hữu sản phẩm và các chuyên gia kỹ thuật để thực hiện các dự án cụ thể.
  - Phân tích thiết kế HTTT



Vai trò củaThiết kế kiến trúc phần mềm

- 1. Quản lý độ phức tạp (Complexity Management)
  - giúp chia nhỏ hệ thống thành các (components) và module độc lập.
  - giúp hiểu rõ bức tranh tổng thể, tránh bị "ngợp" bởi các chi tiết kỹ thuật.
- 2. Đảm bảo các thuộc tính chất lượng (Quality Attributes)
  - System Design thường tập trung vào "làm sao để chạy được",
  - Architecture tập trung vào các tiêu chuẩn:Scalability,Security, Maintainability
- 3. Tối ưu hóa chi phí và nguồn lực
  - giúp phát hiện sớm các rủi ro, đưa ra các quyết định chọn công nghệ phù hợp,
  - tránh lãng phí vào những giải pháp không tương thích
- 4. Tạo ngôn ngữ chung (Communication Tool)
- 5. Hướng dẫn cho thiết kế chi tiết (Blueprinting)
  - thực hiện System Design (chọn database , API design , cache ...) trở nên nhất quán và bám sát mục tiêu Khi có một kiến trúc, .

- Dẫn dắt nhóm phát triển và phối hợp các nỗ lực phát triển thiết kế.
- Có khả năng xem xét các đề xuất thiết kế và thỏa hiệp .

- Chuyên môn về lĩnh vực
  - Chuyên gia về hệ thống được phát triển và kế hoạch phát triển.
  - Hỗ trợ quá trình điều tra yêu cầu, đảm bảo tính đầy đủ và nhất quán.
  - Phối hợp định nghĩa mô hình miền cho hệ thống đang được phát triển.
- Chuyên môn công nghệ
  - Chuyên gia về các công nghệ hiện có giúp triển khai hệ thống.
  - Phối hợp lựa chọn ngôn ngữ lập trình, khuôn khổ, nền tảng, cơ sở dữ liệu..
- Chuyên môn phương pháp luận
  - Chuyên gia về phương pháp phát triển phần mềm
  - Chọn phương pháp phát triển phù hợp giúp ích cho toàn bộ nhóm.

Hạn chế

- Kiến trúc phần mềm vẫn là một ngành mới nổi trong kỹ thuật phần mềm.
  - Thiếu nhận thức về tầm quan trọng của thiết kế kiến trúc đối với phát triển phần mềm.
  - Thiếu hiểu biết về vai trò của kiến trúc sư phần mềm và giao tiếp kém giữa các bên liên quan.
  - Thiếu công cụ và phương pháp chuẩn hóa để thể hiện kiến trúc.
  - Thiếu phương pháp phân tích để dự đoán liệu kiến trúc có tạo ra được sự triển khai đáp ứng các yêu cầu hay không.
  - Thiếu hiểu biết về quy trình thiết kế, kinh nghiệm thiết kế và đánh giá thiết kế.

Software Architecture Parttern

Bốn Chức năng Cơ bản của Phần mềm

- "Khi thiết kế kiến trúc, không chỉ nghĩ về một khối phần mềm duy nhất.
  - Mọi ứng dụng đều có thể được chia thành 4 góc nhìn cơ bản.
  - Cách phân bổ các thành phần này lên phần cứng sẽ quyết định loại hình sơ đồ kiến trúc."
- 4 góc nhìn Phần mềm:
  1. Logic Trình bày (Presentation logic): Giao diện người dùng (UI).
  2. Logic Ứng dụng (Application logic): Các quy tắc nghiệp vụ (Business rules).
  3. Logic Truy cập Dữ liệu (Data access logic): Xử lý truy vấn, ví dụ SQL.
  4. Lưu trữ Dữ liệu (Data storage): Dữ liệu trong cơ sở dữ liệu hoặc tệp.
- 1 góc nhìn vật lý (Physical architecture)
  - quyết định cách 4 chức năng này được phân bổ trên các thành phần phần cứng khác nhau

Khái niệm Parttern

- Việc phân bổ 4 chức năng lên các phần cứng khác nhau tạo ra nhiều loại hình kiến trúc.
- Ví dụ
  - Server-Based:
    - Toàn bộ 4 chức năng nằm trên một máy chủ.
    - Client chỉ là thiết bị đầu cuối (terminal).
  - Client-Based:
    - Hầu hết các chức năng nằm trên máy khách.
    - Máy chủ chỉ làm nhiệm vụ lưu trữ dữ liệu.
  - Client-Server:
    - Logic được phân chia giữa client và server.
    - Đây là kiến trúc phổ biến nhất hiện nay, cho phép cân bằng tải và tăng khả năng mở rộng.
- Các giải pháp kiến trúc này
  - có thể tái sử dụng cho các vấn đề phổ biến trong thiết kế cấu trúc và tương tác của các hệ thống phần mềm.
  - Trở thành Parttern nhằm giảm thời gian và chi phí phát triển

Software Architecture Patterns - mẫu kiến trúc phần mềm

- Software architecture pattern, Wikipedia,
  - là một giải pháp chung, có thể tái sử dụng để giải quyết một vấn đề thường gặp trong một hoàn cảnh cụ thể trong kiến trúc phần mềm.
  - Nó giống như một khuôn mẫu có thể được áp dụng để tạo ra kiến trúc của một hệ thống phần mềm.
- Các mẫu kiến trúc, thường được gọi là phong cách kiến trúc,
  - tạo ra những giải pháp lặp đi lặp lại cho các vấn đề thường gặp trong thiết kế phần mềm, được chứng minh và có thể tái sử dụng
- Chúng vượt ra ngoài những đặc thù của các công nghệ cụ thể
  - cung cấp khuôn mẫu cấp cao để tổ chức và cấu trúc các hệ thống phần mềm.
- Chúng là các mẫu hình là thực tiễn tốt nhất,
  - cung cấp cho các nhà phát triển một tập hợp các quy tắc và hướng dẫn
  - để tạo ra các giải pháp phần mềm có khả năng mở rộng, dễ bảo trì và hiệu quả.

(Monolithic Architecture) và Master-Slave

- Kiến trúc nguyên khối (Monolithic Architecture)
  - cách tiếp cận trong đó tất cả các thành phần của ứng dụng được tích hợp chặt chẽ thành một đơn vị duy nhất, gắn kết.
  - Phù hợp với các dự án nhỏ hoặc những dự án có yêu cầu đơn giản,
  - toàn bộ ứng dụng được coi là một đơn vị duy nhất, không thể chia tách.
  - Tất cả các yêu cầu đều được xử lý trong đơn vị này
- Kiến trúc Master-Slave
  - là một mô hình phân tán, trong đó thực thể trung tâm, được gọi là (master node), điều khiển và phân công nhiệm vụ cho các thực thể phụ thuộc được gọi là (slave node).
  - Kiến trúc này đặc biệt hữu ích trong các ứng dụng xử lý dữ liệu lớn và hệ thống điện toán phân tán

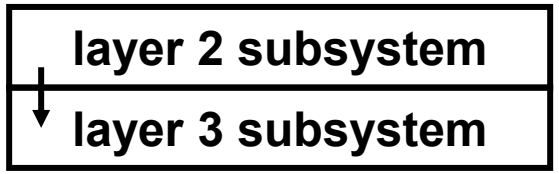
Mẫu kiến trúc MVC

- Model-View-Controller (MVC)
  - là một mô hình kiến trúc lâu đời trong phát triển web và các ứng dụng giao diện người dùng đồ họa (GUI),
  - Mô hình kiến trúc này tăng cường tính mô-đun, giúp dễ dàng mở rộng và bảo trì ứng dụng theo thời gian.
- Ưu điểm:
  - Phân tách vai trò, chức năng cho từng loại lớp
  - Dễ dàng phân công phát triển và kiểm thử
  - sự tách biệt MVC này cho phép cắm các ứng dụng khách front-end khác nhau và cũng cung cấp sự tách biệt trong quá trình phát triển

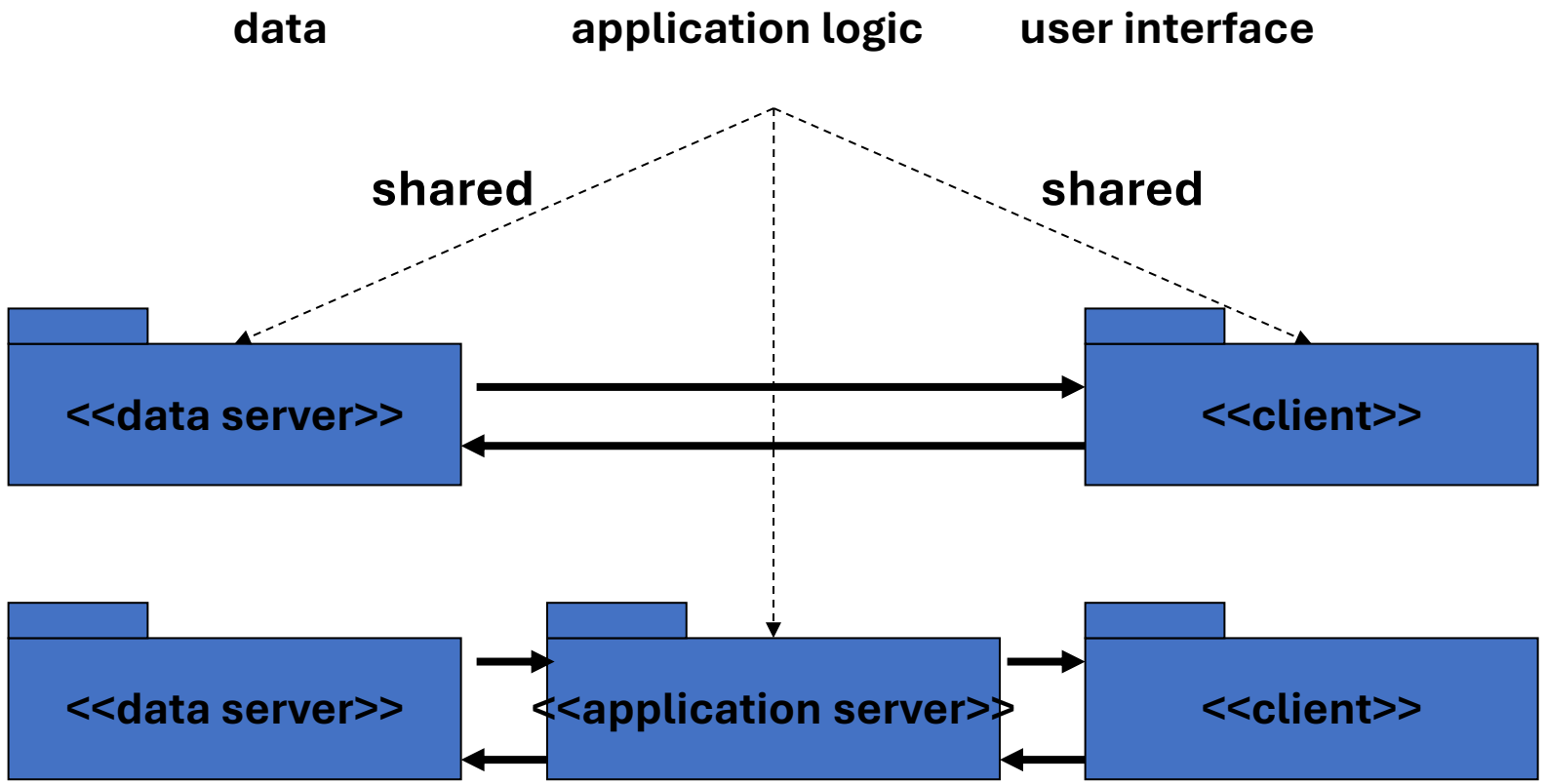
Kiến trúc Đa tầng (Client-Server Tiers)

- "Kiến trúc Client-Server không chỉ có hai thành phần.
  - Để tăng tính linh hoạt và khả năng mở rộng, người ta thường chia nhỏ nó thành nhiều tầng (tiers) hơn, mỗi tầng đảm nhiệm một chức năng cụ thể.“
- Chi tiết:
  - Two-Tiered Architecture: client và server.
  - Three-Tiered Architecture:
    - Client (Presentation), Application Server (Application Logic), Database Server (Data Access & Storage).
  - N-Tiered Architecture: Mở rộng từ 3 tầng,
    - App logic được phân bổ trên nhiều server khác nhau.
    - Phổ biến trong các hệ thống e-commerce.

2-tier and 3-tier architecture



Layer 2 sử dụng
Dịch vụ của lớp 3

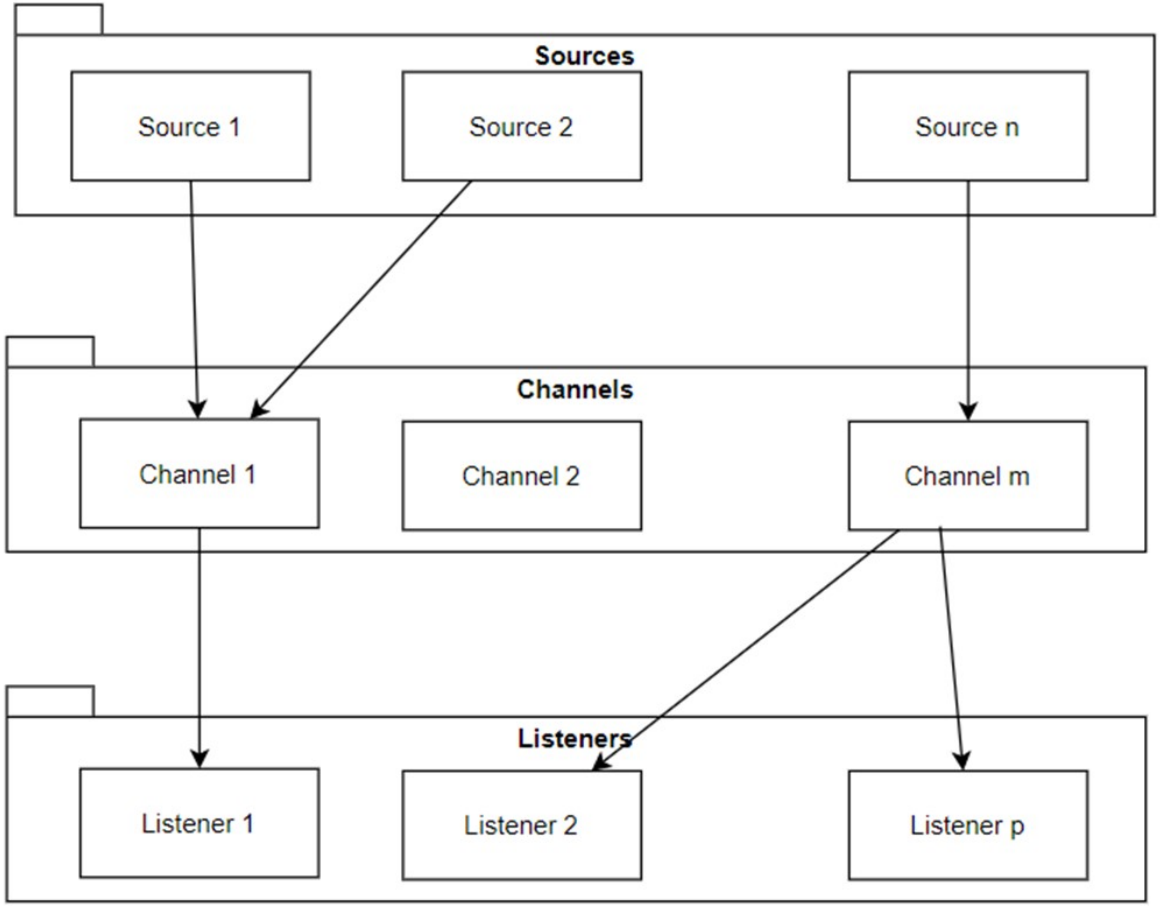


Kiến trúc hướng sự kiện Event-Driven Architecture - EAD)

- Event-Driven Architecture - EAD)
  - Một mô hình thiết kế phần mềm trong đó các hệ thống giao tiếp bằng cách tạo ra và phản hồi các "sự kiện".
  - Sự kiến :
    - những thay đổi trạng thái riêng biệt như khách hàng đặt hàng hoặc cảm biến phát hiện sự tăng đột biến nhiệt độ.
- EDA hoạt động **bất đồng bộ** và **tách rời**
  - Trái mô hình yêu cầu-phản hồi truyền thống, nơi hệ thống chờ đợi phản hồi,
  - một dịch vụ có thể phát ra một sự kiện mà không cần biết dịch vụ nào khác sẽ tiêu thụ nó,
    - cho phép khả năng mở rộng cao và phản hồi theo thời gian thực

Mẫu kiến trúc Event

- Phân chia hệ thống thành các lớp phát sinh sự kiện (source) và các lớp xử lý sự kiện (listener).
- Sự kiện được phân luồng theo các kênh (channel).
- Ưu điểm
  - Đơn giản hóa các logic phức tạp của ứng dụng
  - Phân tách giữa các yếu tố phát sinh sự kiện và các yếu tố xử lý sự kiện



Ưu điểm

- 1. Khả năng mở rộng vượt trội (Scalability)
  - Các thành phần trong EDA hoạt động độc lập.
  - Khi lượng dữ liệu tăng đột biến, chỉ cần mở rộng đúng bộ phận xử lý sự kiện đó
- 2. Giảm thiểu sự phụ thuộc (Loose Coupling)
  - Nếu một dịch vụ gặp sự cố hoặc đang bảo trì, các dịch vụ khác vẫn chạy .
- 3. Phản hồi theo thời gian thực (Real-time Responsiveness)
  - quan trọng cho các ứng dụng: phát hiện gian lận ngân hàng, thông báo đơn hàng hoặc cập nhật vị trí shipper.
- 4. Khả năng hồi phục cao (Resilience)
  - Nếu một dịch vụ xử lý bị sập, các sự kiện vẫn nằm an toàn trong hàng đợi (queue) không bị mất dữ liệu.

Kiến trúc hướng dịch vụ (Service-Oriented Architecture - SOA)

- SOA
  - là một phương pháp thiết kế phần mềm trong đó các thành phần của ứng dụng được xây dựng thành các dịch vụ (services) riêng biệt, có khả năng giao tiếp với nhau qua mạng
- Đặc điểm
  - Tính lỏng lẻo (Loosely coupled):
    - Các dịch vụ hoạt động độc lập, liên kết lỏng lẻo ;
    - thay đổi bên trong một dịch vụ không làm ảnh hưởng đến các dịch vụ khác
  - Mỗi dịch vụ cung cấp chức năng của mình thông qua các giao thức tiêu chuẩn.
  - SOA thúc đẩy khả năng tái sử dụng,
    - cho phép dịch vụ được sử dụng trong nhiều ứng dụng khác nhau
- SOA thường được sử dụng trong các ứng dụng cấp doanh nghiệp

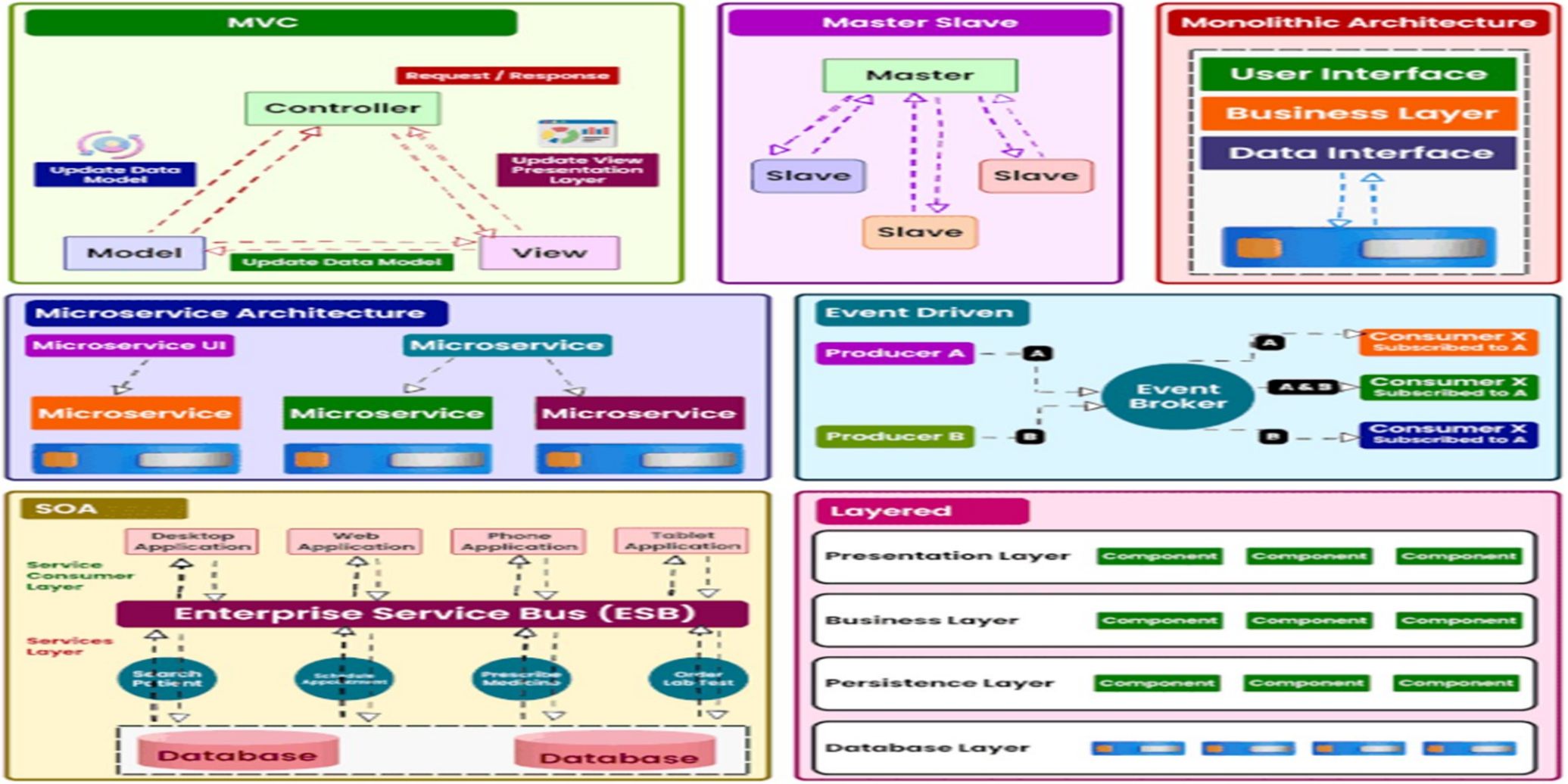
Mẫu kiến trúc Microservices

- Kiến trúc Microservices
  - là một phương pháp hiện đại phân tách ứng dụng thành một tập hợp các dịch vụ nhỏ, độc lập.
  - Mỗi dịch vụ chạy quy trình riêng và giao tiếp với các dịch vụ khác thông qua API.
- Ưu điểm
  - Độc lập trong phát triển
  - Độc lập trong triển khai và tích hợp
  - Kích thước nhỏ, dễ phát triển và kiểm thử
  - Dễ cô lập và thay thế
- Microservices: Là một dạng tiến hóa của SOA,
  - tập trung vào việc chia nhỏ một ứng dụng đơn lẻ thành dịch vụ nhỏ để triển khai nhanh và liên tục

Các Xu hướng Kiến trúc khác

- "Công nghệ không ngừng phát triển, và kiến trúc hệ thống cũng vậy.
  - Các xu hướng như điện toán đám mây và Internet Vạn vật đang định hình lại cách chúng ta thiết kế và triển khai hệ thống."
- Cloud Computing:
  - Xem IT như một dịch vụ tiện ích, tài nguyên được cung cấp qua Internet.
  - Các loại dịch vụ: IaaS, PaaS, SaaS.
- Ubiquitous Computing và Internet of Things (IoT):
  - Ý tưởng máy tính có ở khắp mọi nơi và trong mọi vật dụng, tạo ra các "vật thể mê hoặc" (enchanted objects).

Kiến trúc

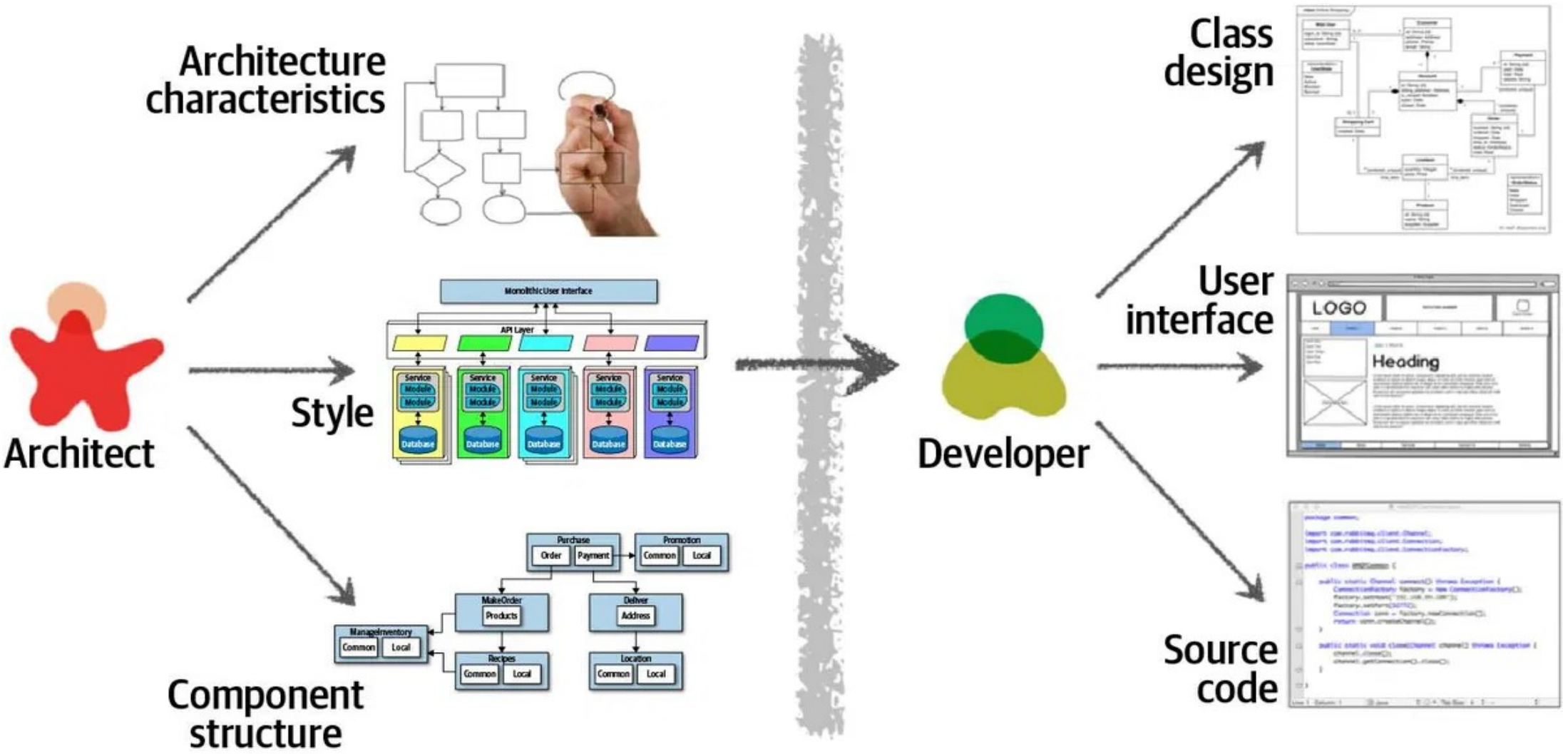

MVC
Request / Response
Controller
Update Data Model
Update View Presentation Layer
Model
Update Data Model
View
Master Slave
Master
Slave
Slave
Slave
Monolithic Architecture
User Interface
Business Layer
Data Interface
Microservice Architecture
Microservice UI
Microservice
Microservice
Microservice
Microservice
Event Driven
Producer A
Producer B
Event Broker
Consumer X Subscribed to A
Consumer X Subscribed to A
Consumer X Subscribed to A
SOA
Desktop Application
Web Application
Phone Application
Tablet Application
Service Consumer Layer
Enterprise Service Bus (ESB)
Services Layer
Search Patient
Schedule Appointment
Prescribe Medicine
Order Lab Test
Database
Database
Layered
Presentation Layer
Component
Component
Component
Business Layer
Component
Component
Component
Persistence Layer
Component
Component
Component
Database Layer

Architecture pattern vs Design pattern

- Các mẫu kiến trúc
  - cung cấp bản thiết kế cấp cao cho toàn hệ thống hoặc các hệ thống con .
  - "khung sườn" và cách các thành phần khác nhau tương tác với nhau.
- Phạm vi:
  - Toàn hệ thống.
- Trọng tâm:
  - Tổ chức cấu trúc, quy tắc giao tiếp giữa các mô-đun và khả năng mở rộng và bảo mật.
- Thời điểm:
  - Được quyết định trong giai đoạn lập kế hoạch ban đầu.

- Các mẫu thiết kế
  - là những giải pháp cục bộ, tái sử dụng cho các vấn đề lập trình phổ biến.
  - tập trung vào các chi tiết triển khai bên trong các lớp hoặc mô-đun cụ thể.
- Phạm vi:
  - Giới hạn trong các phần mã .
- Trọng tâm:
  - Tương tác giữa các đối tượng và lớp để giải quyết một thách thức lập trình cụ thể.
- Thời điểm áp dụng:
  - Trong giai đoạn phát triển/triển khai.

Architecture pattern vs Design pattern



Sơ đồ biểu diễn Architecture Diagrams C4 - UML

Sơ đồ kiến trúc - Architecture Diagrams

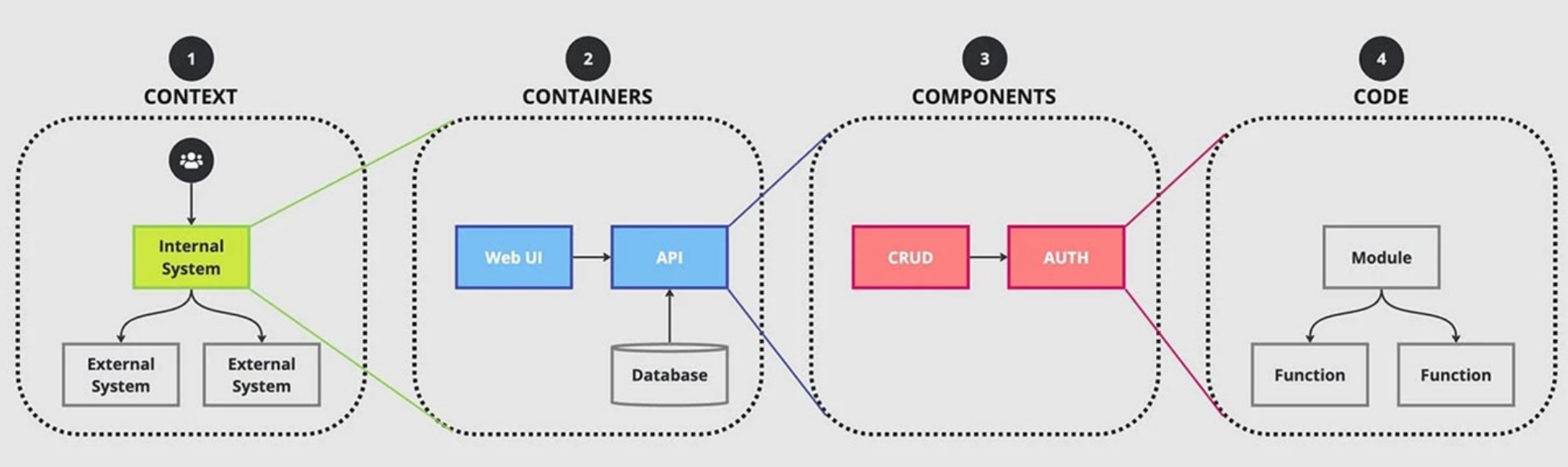
- Architecture Diagram
  - Bản vẽ trực quan của một hệ thống (phần mềm, CNTT,..), biểu diễn trừu tượng về kiến trúc thành phần của hệ thống.
  - Nó cung cấp một mô tả ngắn gọn về thành phần hệ thống, mối liên hệ ,chức năng, luồng dữ liệu mà mỗi thành phần sử dụng.
- Chúng hoạt động như một bản đồ
  - giúp hiểu, thiết kế, xây dựng và bảo trì các cấu trúc phức tạp,
  - tương tự như cách bản vẽ xây dựng thể hiện các phòng, đường ống và dây điện.
- Các thành phần và mối quan hệ khác nhau tạo ra sơ đồ khác nhau,
  - mô tả khác nhau đáp ứng các góc nhìn khác nhau của nhà phát triển.

C4 Model - Simon Brown,

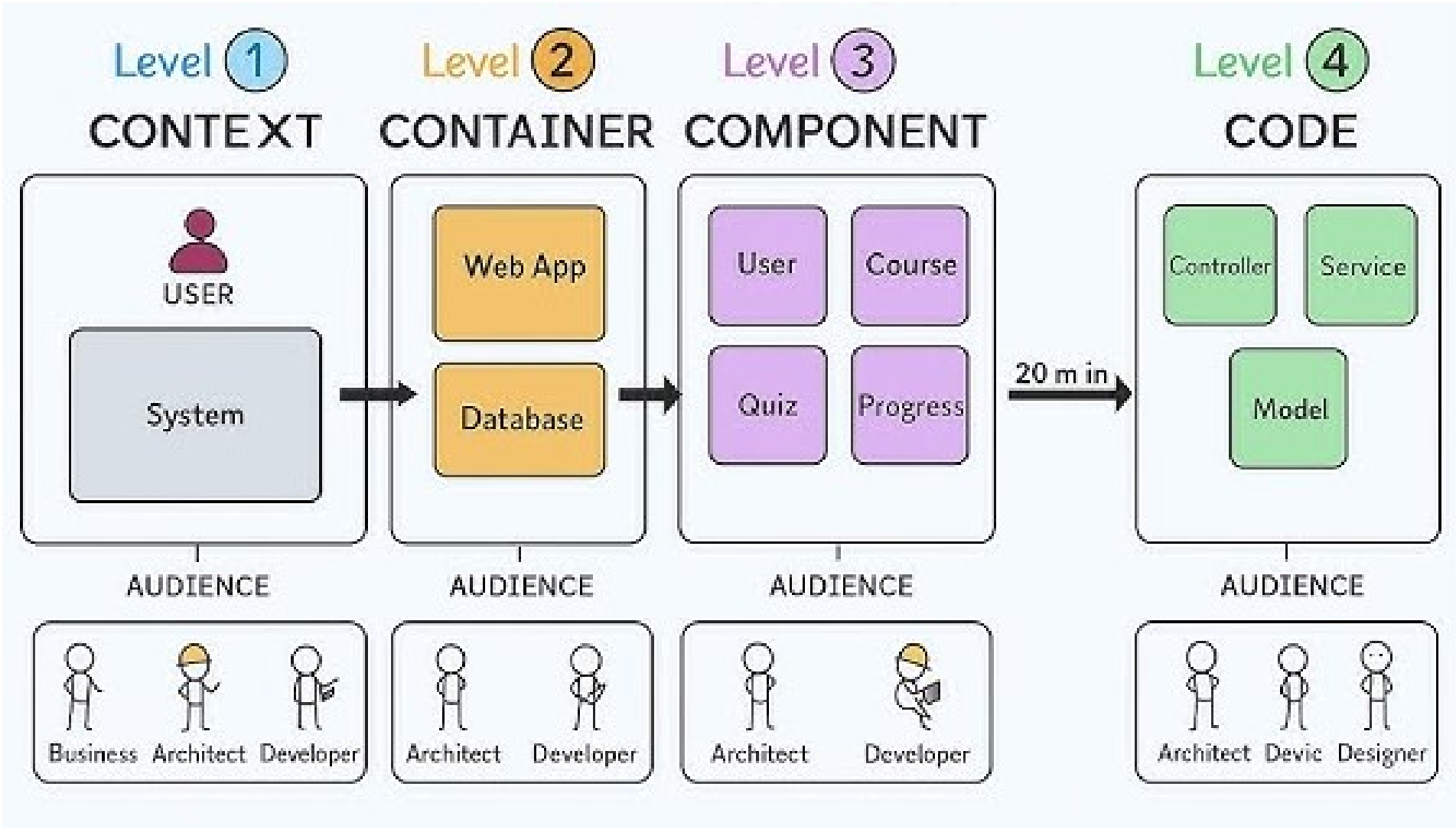
- Kiến trúc phần mềm được biểu diễn bằng nhiều loại sơ đồ khác nhau,
  - tùy thuộc vào mục đích và đối tượng mà kiến trúc hướng tới.
  - chủ yếu sử dụng (UML), ArchiMate, SysML hoặc mô hình C4,
- Mô hình C4
  - Từ một tập hợp các thói quen vẽ sơ đồ cá nhân đơn giản, đã phát triển thành một tiêu chuẩn công nghiệp toàn cầu để trực quan hóa các hệ thống phần mềm.
- C4 là một phương pháp dễ học, thân thiện với nhà phát triển để lập sơ đồ kiến trúc phần mềm
  - không phụ thuộc Ký hiệu.
  - Không phụ thuộc ngôn ngữ lập trình
  - Không phụ thuộc vào công cụ hỗ trợ .

Thành phần cơ bản 4C (Context, Containers, Components, Code)

- C4 cung cấp một phương pháp phân cấp để lập tài liệu kiến trúc thông qua bốn cấp độ chính:
  - ngữ cảnh hệ thống, vùng chứa, thành phần và mã .
- Mỗi cấp độ sẽ "phóng to" để cung cấp thêm chi tiết kỹ thuật cho các đối tượng khác nhau.

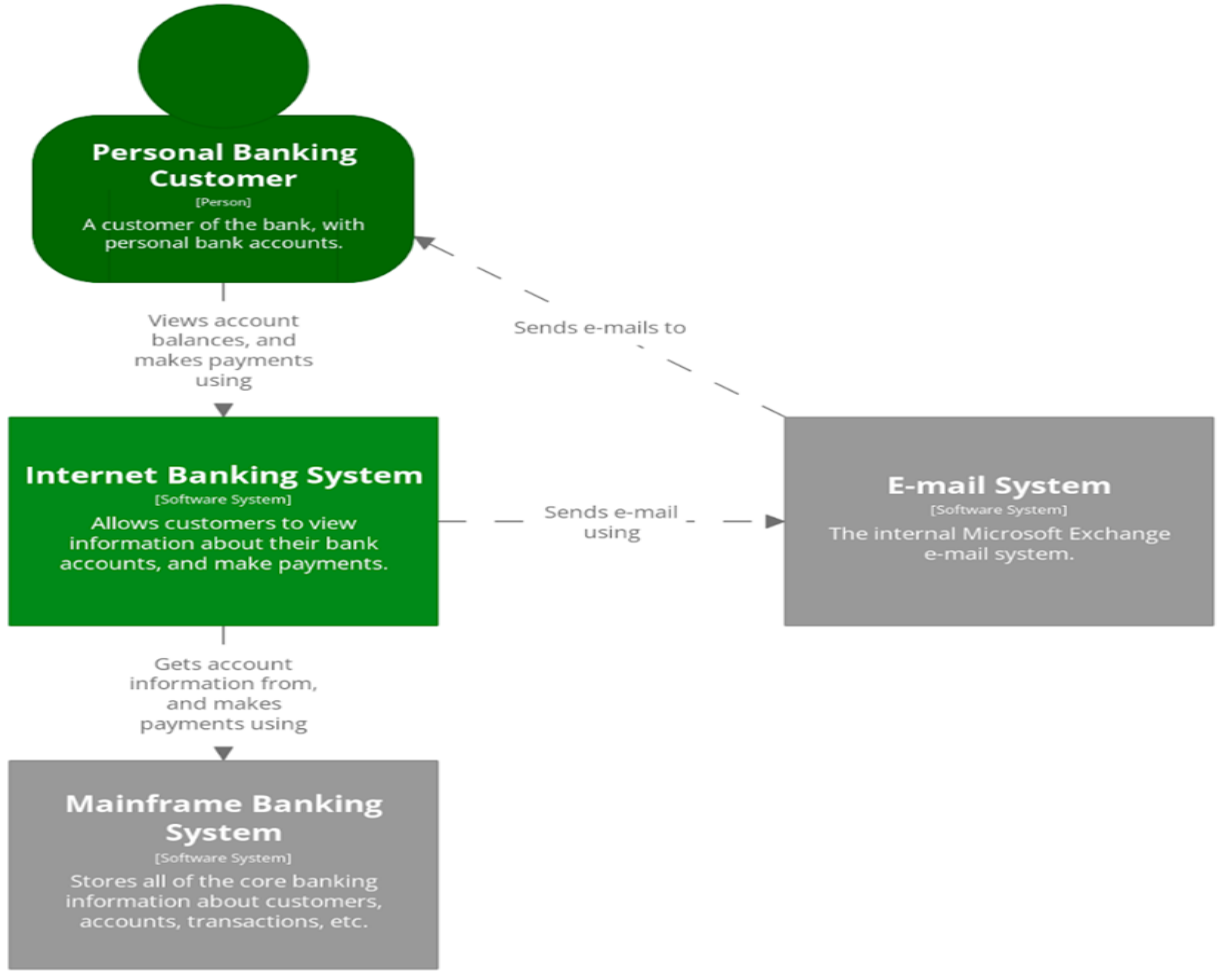


Mô hình này phân rã hệ thống qua 4 cấp độ



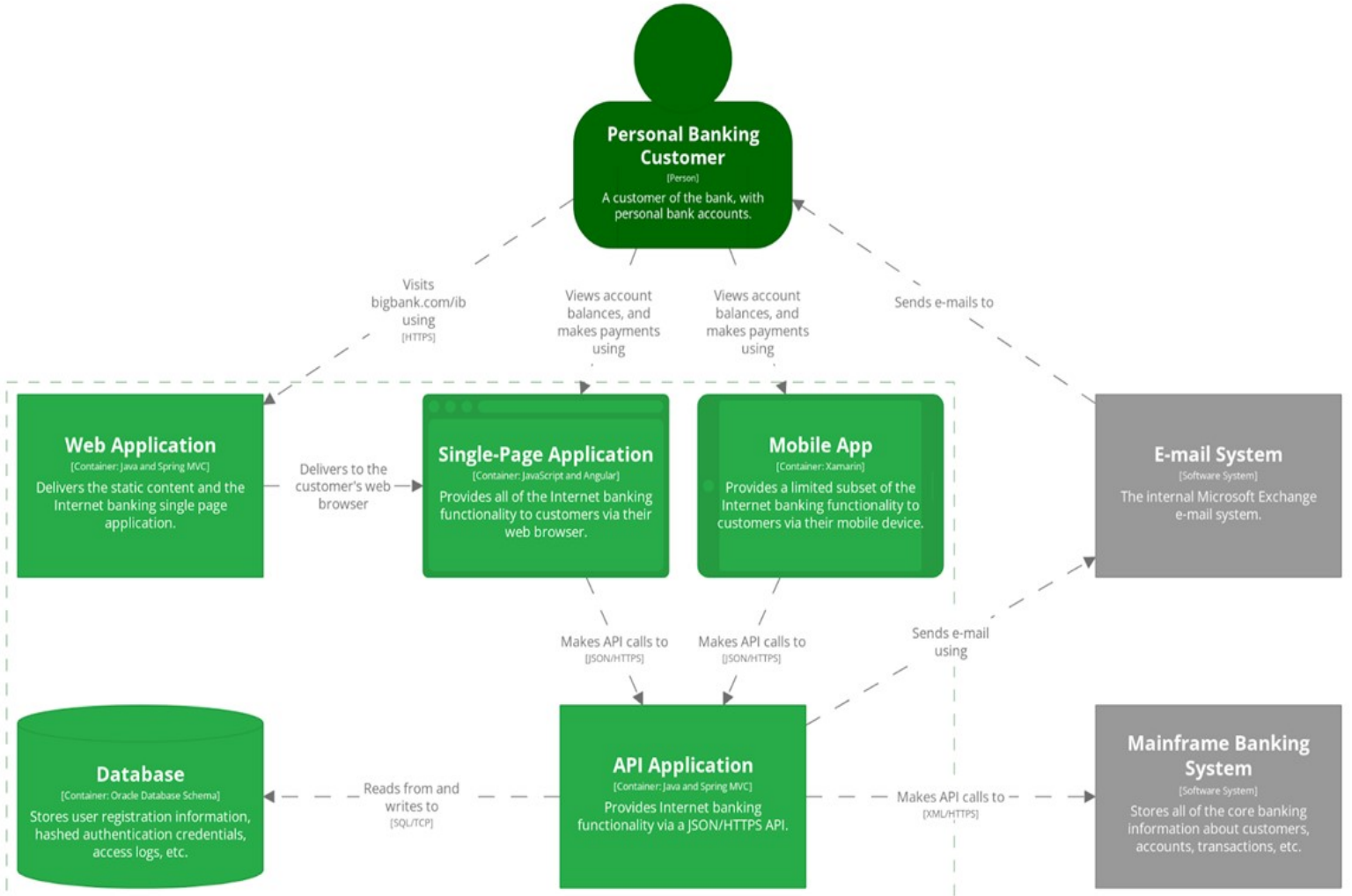
C1: System Context Bối cảnh hệ thống

- Bối cảnh hệ thống
  - bức tranh tổng quát, tập trung vào cách hệ thống tương tác với thế giới bên ngoài..
  - Giải pháp dành cho người không chuyên về kỹ thuật)
- Thành phần
  - Người dùng (Person):
    - Các đối tượng sử dụng hệ thống
    - (ví dụ: Khách hàng, Quản trị viên).
  - (Software System):
    - Hệ thống và các hệ thống bên ngoài liên quan
    - (ví dụ: Cổng thanh toán, Hệ thống gửi Email).
  - Mối quan hệ:
    - Cách các đối tượng và hệ thống trao đổi dữ liệu với nhau.



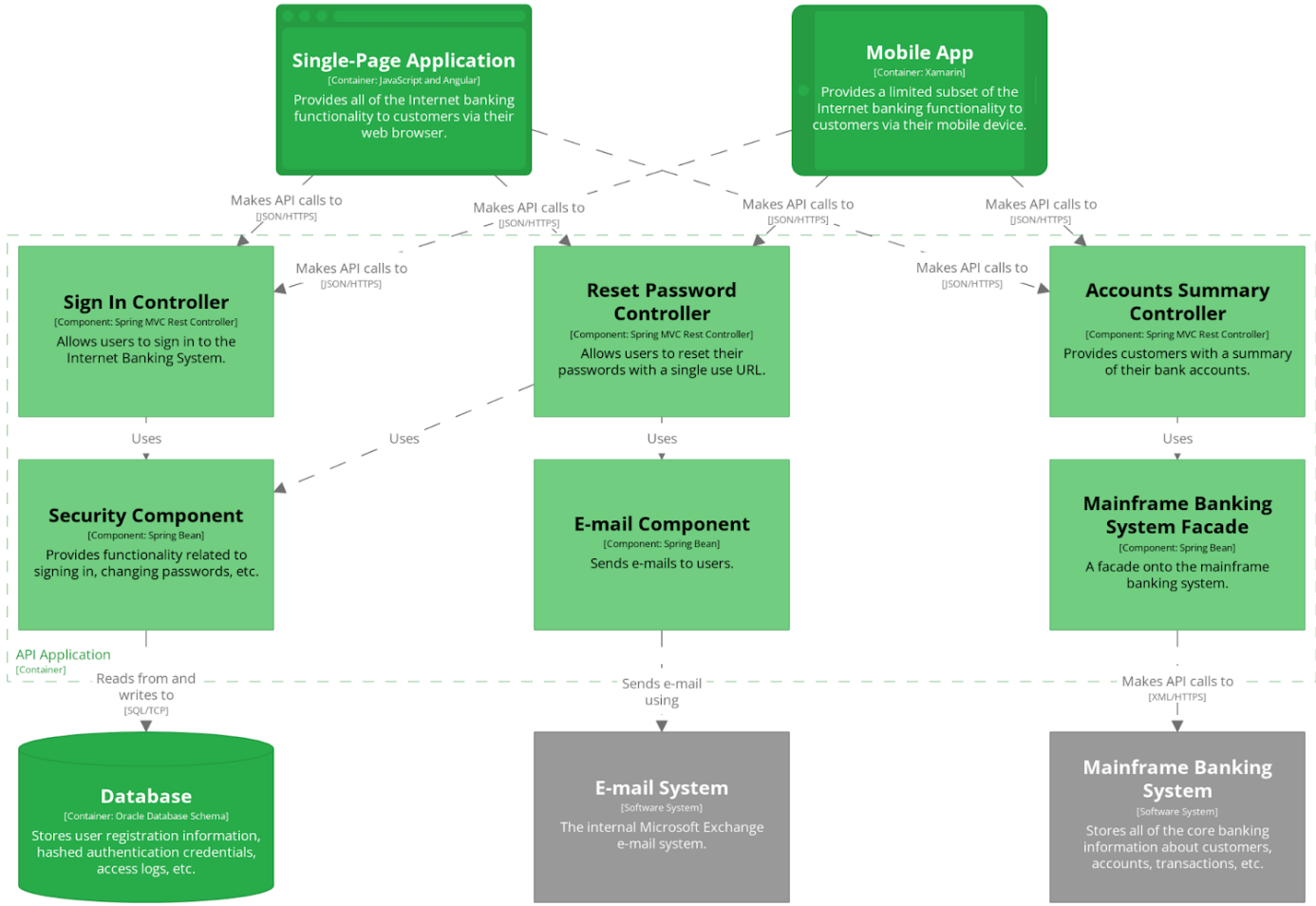
C2: Containers Thành phần thực thi

- Containers
  - Cấp độ này đi sâu vào bên trong hệ thống để xem các đơn vị triển khai/chạy độc lập.
- Ví dụ
  - Web Application:
    - Ứng dụng phía người dùng.
  - Mobile App:
    - Ứng dụng di động.
  - API Application:
    - Dịch vụ phía máy chủ (Backend).
  - Database:
    - Các loại cơ sở dữ liệu hoặc hệ thống lưu trữ.
  - Mối quan hệ:
    - Các giao thức giao tiếp giữa các container
    - (ví dụ: REST API qua HTTP, SQL qua JDBC).



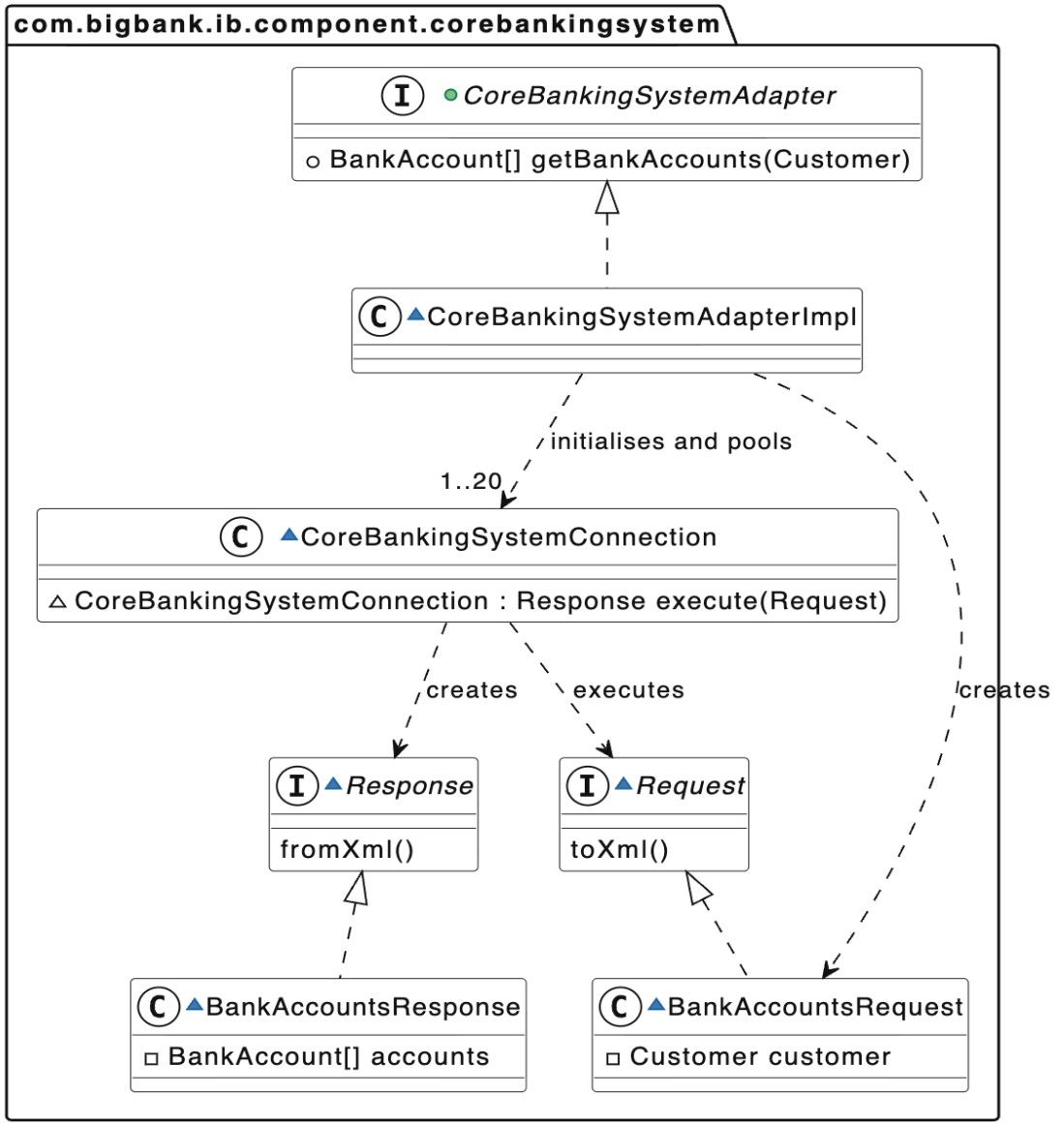
C3 Component Diagram - Thành phần cấu trúc

- C3: Components
  - Trình bày chi tiết hơn về các thành phần riêng lẻ bên trong một vùng chứa,
  - Phân rã cụ thể thành các thành phần logic bên trong.
- Vd Module/Component
  - -Các khối chức năng như
    - "Security Component",
    - "Payment Controller",
    - "Email Service".
  - Mối quan hệ:
    - Cách các thành phần gọi nhau hoặc phụ thuộc lẫn nhau.

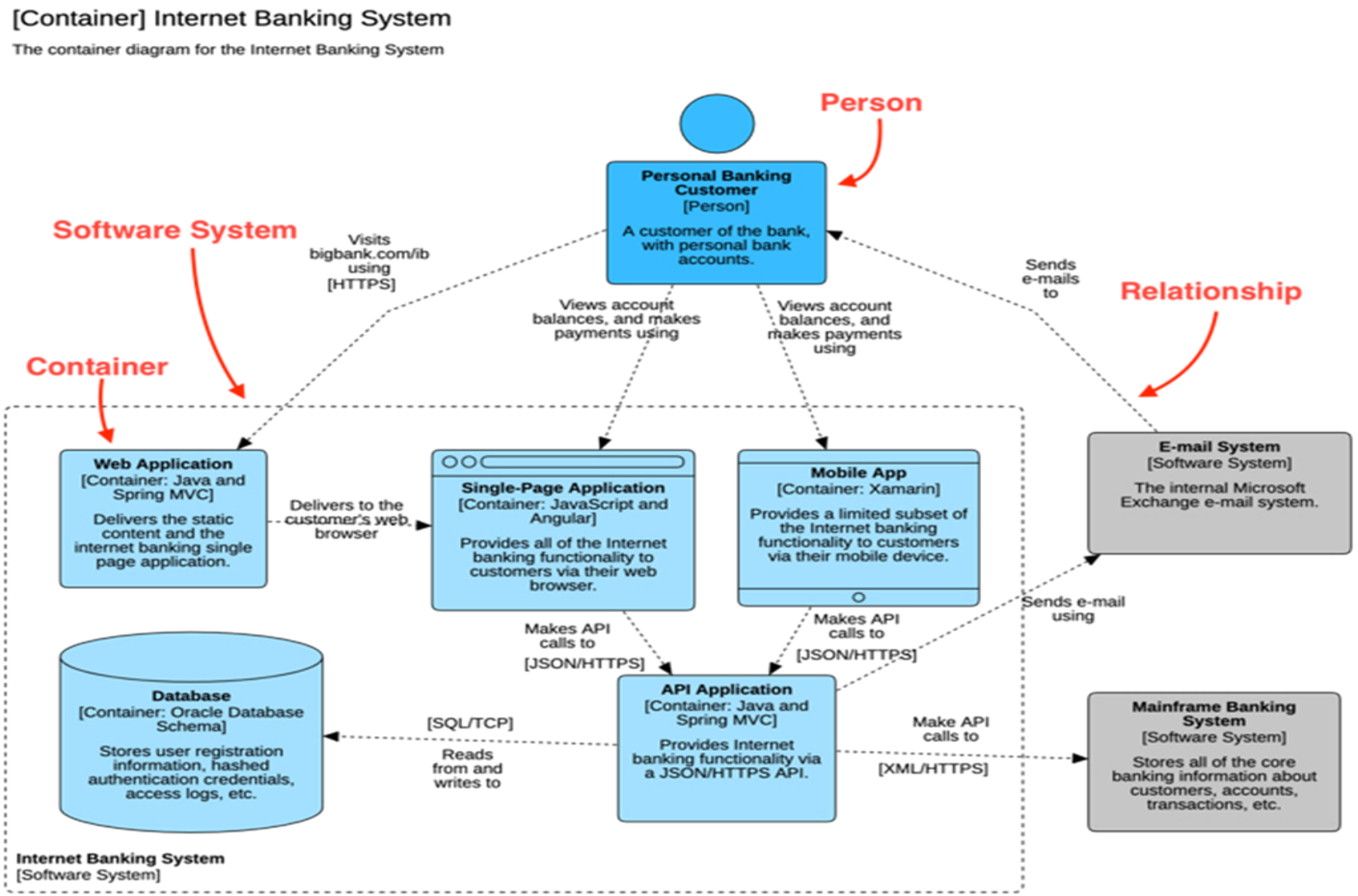


C 4 Code Diagram

- C4: Code (Mã nguồn) là mức chi tiết nhất, thường được biểu diễn bằng
  - biểu đồ lớp (Class Diagram) hoặc sơ đồ thực thể (ER Diagram).
    - Đối tượng: Class, Interface, Function.
    - Mối quan hệ: Kế thừa, thực thi, phụ thuộc.
- Lưu ý:
  - Cấp độ này thường không bắt buộc và chỉ nên vẽ cho các phần cực kỳ phức tạp.
  - Sơ đồ ER và các sơ đồ khác từ bộ sơ đồ UML được sử dụng ở cấp độ cuối cùng của mô hình C4.



Ví dụ


[Container] Internet Banking System
The container diagram for the Internet Banking System
Person
Software System
Container
Relationship
Personal Banking Customer
[Person]
A customer of the bank, with personal bank accounts.
Visits bigbank.com/ib using [HTTPS]
Views account balances, and makes payments using
Views account balances, and makes payments using
Sends e-mails to
Web Application
[Container: Java and Spring MVC]
Delivers the static content and the internet banking single page application.
Delivers to the customer's web browser
Single-Page Application
[Container: JavaScript and Angular]
Provides all of the Internet banking functionality to customers via their web browser.
Mobile App
[Container: Xamarin]
Provides a limited subset of the Internet banking functionality to customers via their mobile device.
E-mail System
[Software System]
The internal Microsoft Exchange e-mail system.
Makes API calls to
[JSON/HTTPS]
Makes API calls to
[JSON/HTTPS]
Sends e-mail using
Database
[Container: Oracle Database Schema]
Stores user registration information, hashed authentication credentials, access logs, etc.
[SQL/TCP]
Reads from and writes to
API Application
[Container: Java and Spring MVC]
Provides Internet banking functionality via a JSON/HTTPS API.
Make API calls to
[XML/HTTPS]
Mainframe Banking System
[Software System]
Stores all of the core banking information about customers, accounts, transactions, etc.
Internet Banking System
[Software System]

Ưu điểm

- Đơn giản và rõ ràng :
  - Sơ đồ C4 được thiết kế để dễ hiểu cho cả nhà phát triển, người quản lý dự án hoặc các bên liên quan không chuyên về kỹ thuật.
- Linh hoạt :
  - chỉ có thể tạo những sơ đồ cần thiết cho dự án của mình
  - (ví dụ: chỉ sơ đồ Ngữ cảnh và Sơ đồ Container).
- Hợp tác nhóm :
  - Mô hình này giúp việc hợp tác giữa các nhóm phân tán dễ dàng hơn bằng cách chuẩn hóa tài liệu.
- Dễ bảo trì :
  - Sơ đồ C4 dễ cập nhật và có thể được triển khai bằng nhiều công cụ khác nhau.

Sơ đồ Kiến trúc trong UML

- "Để trực quan hóa các quyết định kiến trúc,
  - UML cung cấp một bộ công cụ mạnh mẽ giúp mô hình hóa các khía cạnh vật lý và logic của kiến trúc hệ thống."
- Mục tiêu:
  - Trực quan hóa cấu trúc tĩnh của hệ thống.
  - Mô tả cách các thành phần vật lý và logic được tổ chức và triển khai.
- Các sơ đồ UML chính cho kiến trúc:
  1. Package Diagram (Sơ đồ Gói):
     - Dùng để tổ chức các yếu tố mô hình thành các nhóm logic.
  2. Component Diagram (Sơ đồ Thành phần):
     - Mô tả cấu trúc logic của các module phần mềm.
  3. Deployment Diagram (Sơ đồ Triển khai):
     - Mô tả cấu trúc vật lý và cách triển khai phần mềm lên phần cứng.

Nên sử dụng loại nào?

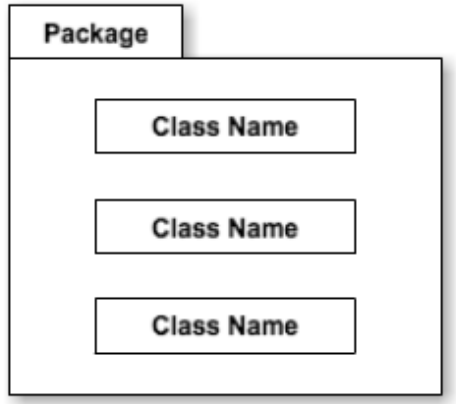
- Tình trạng hiện tại:
  - Mặc dù vẫn là một tiêu chuẩn, nhưng mức độ phổ biến đã giảm sút khi các nhóm ưu tiên các tài liệu "nhẹ nhàng" hơn như C4 để theo kịp phát triển phần mềm theo mô hình Agile.
- C4 để truyền đạt kiến trúc hệ thống cho nhiều đối tượng,
  - hướng dẫn lập trình viên mới hoặc cung cấp cái nhìn tổng quan cấp cao về phạm vi hệ thống.
- UML ghi lại chi tiết các tương tác nội bộ,
  - cấu trúc phân cấp lớp phức tạp hoặc các trạng thái hành vi cụ thể trong một thành phần duy nhất.
- Phối hợp hiệu quả hơn:
  - Nhiều kiến trúc sư hiện đại sử dụng C4 cho các cấp độ 1-3 và
  - chuyển sang Sơ đồ lớp UML hoặc Sơ đồ trình tự khi cần chi tiết ở cấp độ mã nguồn.

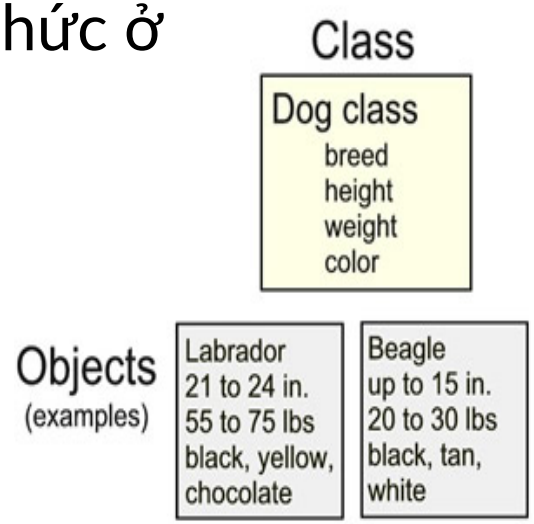
Package Diagram

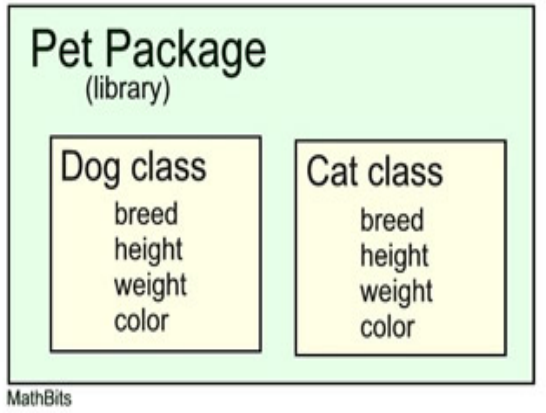
Tại sao cần sơ đồ gói?

- Tại sao cần ?
  - "Khi hệ thống lớn lên, chúng ta cần một cách để 'dọn dẹp' và tổ chức các mô hình.
  - Sơ đồ Gói hoạt động giống như các thư mục trên máy tính, giúp nhóm các yếu tố liên quan (như lớp, use case) lại với nhau để dễ quản lý hơn."
- "Sơ đồ Gói có ký hiệu rất đơn giản,
  - giúp chúng ta tập trung vào cấu trúc tổ chức ở mức cao."







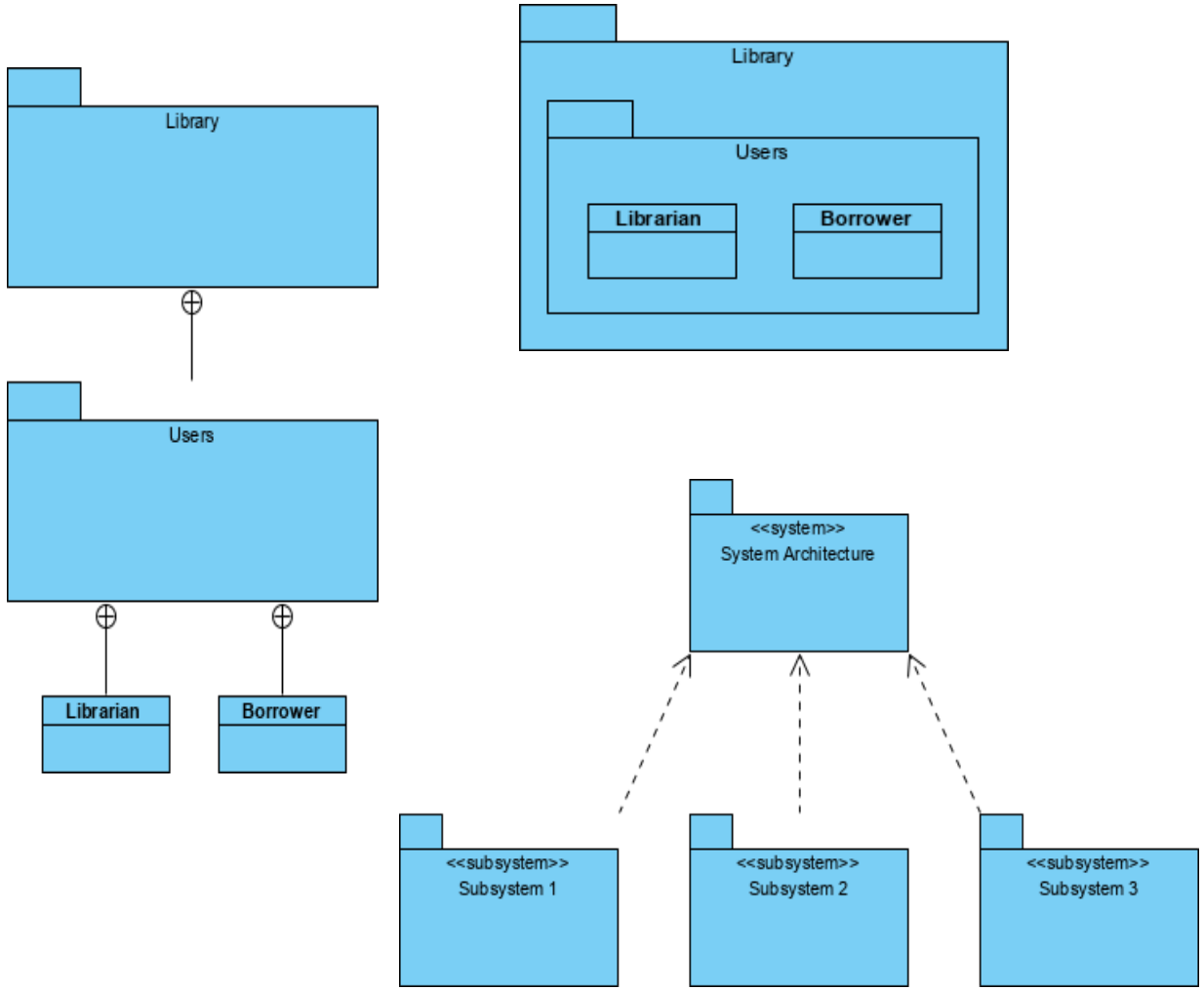


Sơ đồ Gói (Package Diagram)

- Package Diagram
  - sơ đồ cấu trúc được sử dụng để tổ chức các hệ thống quy mô lớn thành các nhóm logic, dễ quản lý được gọi là "gói".
  - Nó hoạt động như một "bản đồ" cấp cao của mã nguồn, cho thấy cách các mô-đun, lớp hoặc hệ thống con khác nhau tương tác với nhau.
- Nó cung cấp cái nhìn tổng quan về cấu trúc của hệ thống bằng cách nhóm các thành phần liên quan, thành các đơn vị logic gọi là "gói".
  - Sơ đồ gói chỉ hiển thị các gói và mối quan hệ phụ thuộc giữa chúng
- Mục đích:
  - Đơn giản hóa các sơ đồ phức tạp (như Sơ đồ lớp).
  - Trực quan hóa các tầng (layers) và phân vùng (partitions) của kiến trúc.

Ký hiệu Sơ đồ Gói

- Ký hiệu:
  - Package: Hình thư mục có tab.
  - Dependency Relationship (Quan hệ Phụ thuộc):
    - Mũi tên nét đứt, chỉ từ gói phụ thuộc đến gói độc lập.
    - Ý nghĩa: thay đổi trong gói độc lập có thể ảnh hưởng đến gói phụ thuộc.
- *Vd:*
  - *Tên gói::tên lớp*
  - *Thư viện::Người dùng::Thủ thư*

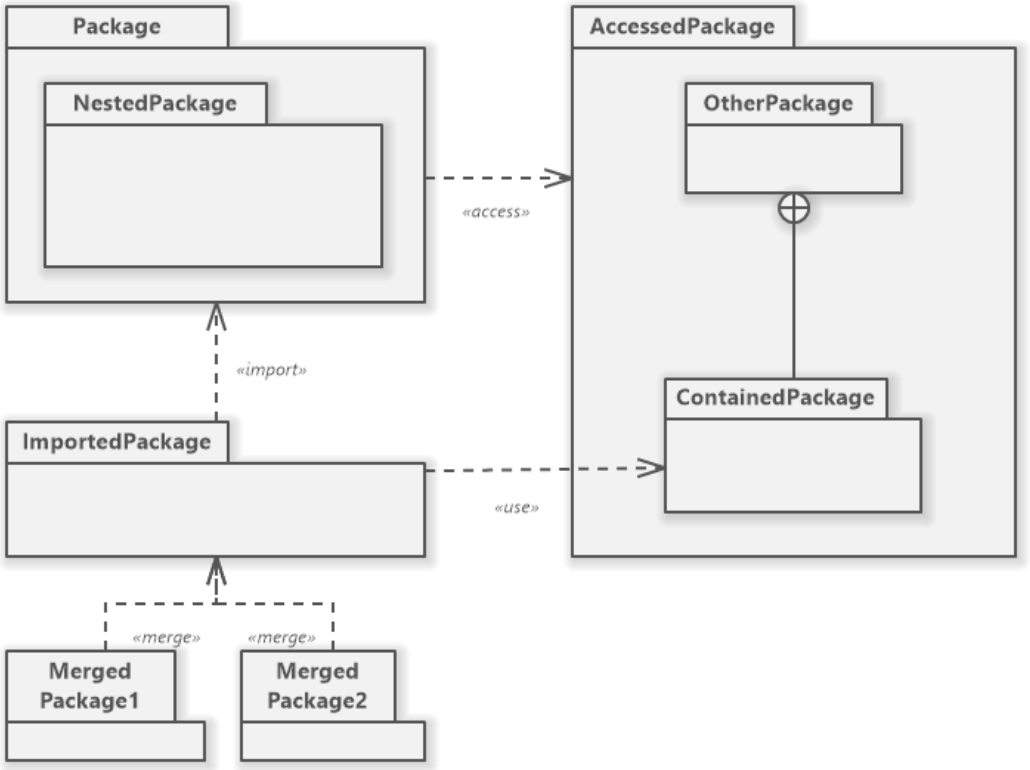


Đặc điểm khi xây dựng gói

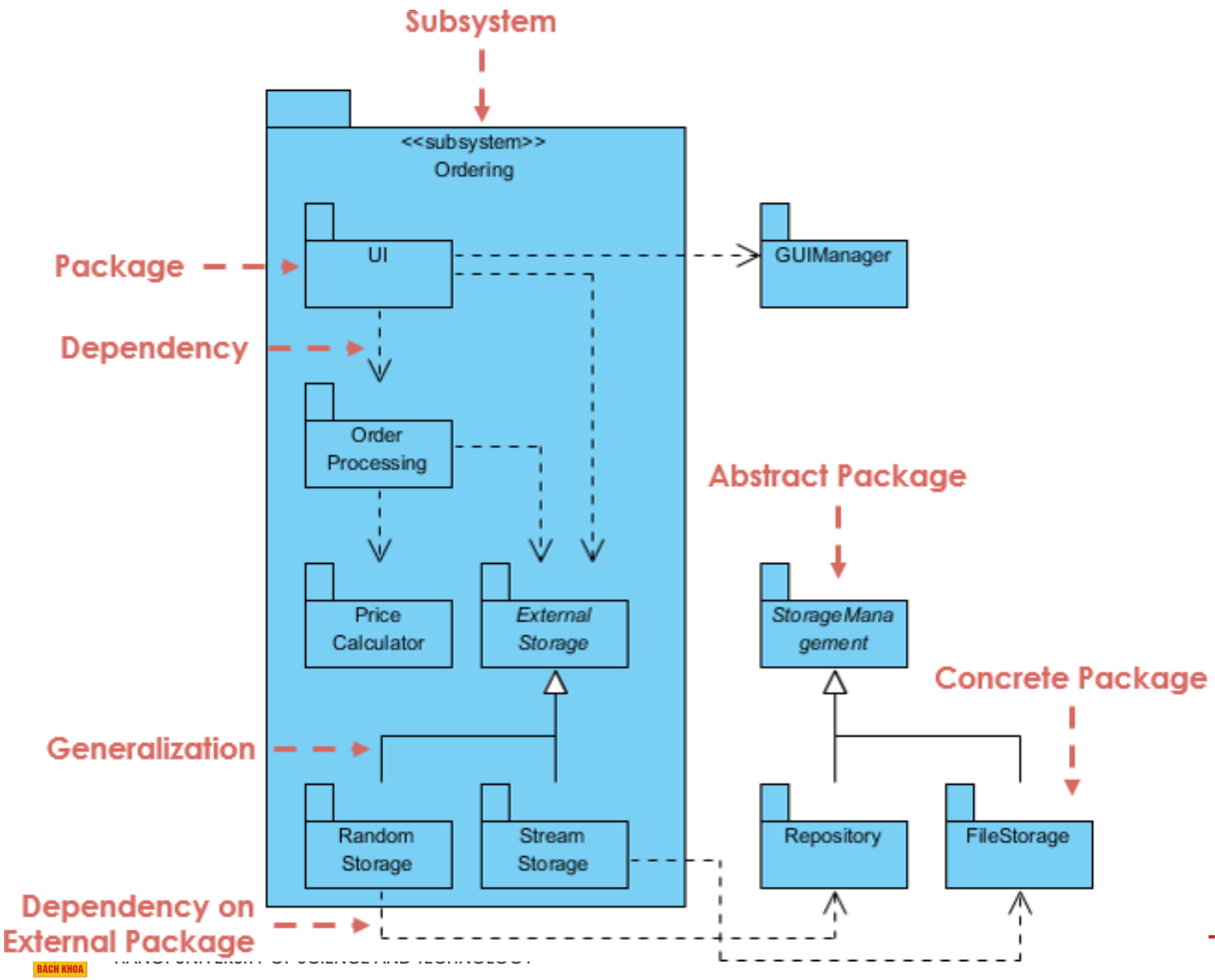
- Về mục đích:
  - Các lớp phải cung cấp các dịch vụ có cùng bản chất cho người dùng.
  - cùng một lĩnh vực và đề cập một số thuật ngữ chung
    - Vd gói giao diện đề cập các thuật ngữ như: cửa sổ, thực đơn, nút nhấn,...)
- Về xu thế phát triển:
  - Người ta tách các lớp bền vững với các lớp có nhiều khả năng thay đổi.
    - tách các lớp nghiệp vụ với các lớp ứng dụng, và xếp chúng vào các hệ con khác nhau.
- Về ứng dụng các công nghệ:
  - tận dụng các công nghệ có sẵn, như các thư viện chương trình (lớp/thành phần), các GUI, các hệ quản trị cơ sở dữ liệu,...,
    - Vd tách các gói giao tiếp, gói quản trị dữ liệu ra khỏi phần lõi (ứng dụng và nghiệp vụ) của hệ thống

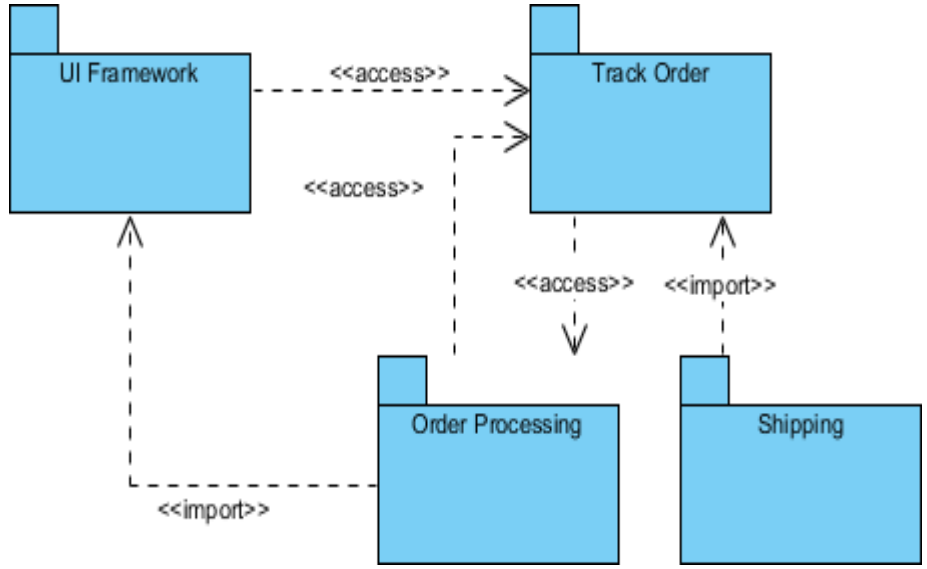
Sơ đồ gói - Ký hiệu phụ thuộc (dependency)

- <<import>> và <<access>> là hai loại quan hệ phụ thuộc giữa các package.
  - <<import>> (Package Import)
    - Thêm các phần tử của 1 package vào package khác.
    - Có tính lan truyền (nếu Package A import B, và B import C, thì A có thể thấy C tùy vào quyền truy cập).
  - <<access>>.
    - package nguồn sử dụng các phần tử của package đích, nhưng **không** đưa chúng vào namespace của mình.
    - Giúp giữ cho namespace của package nguồn sạch sẽ



Ví dụ: Mô hình hóa nhóm phức tạp

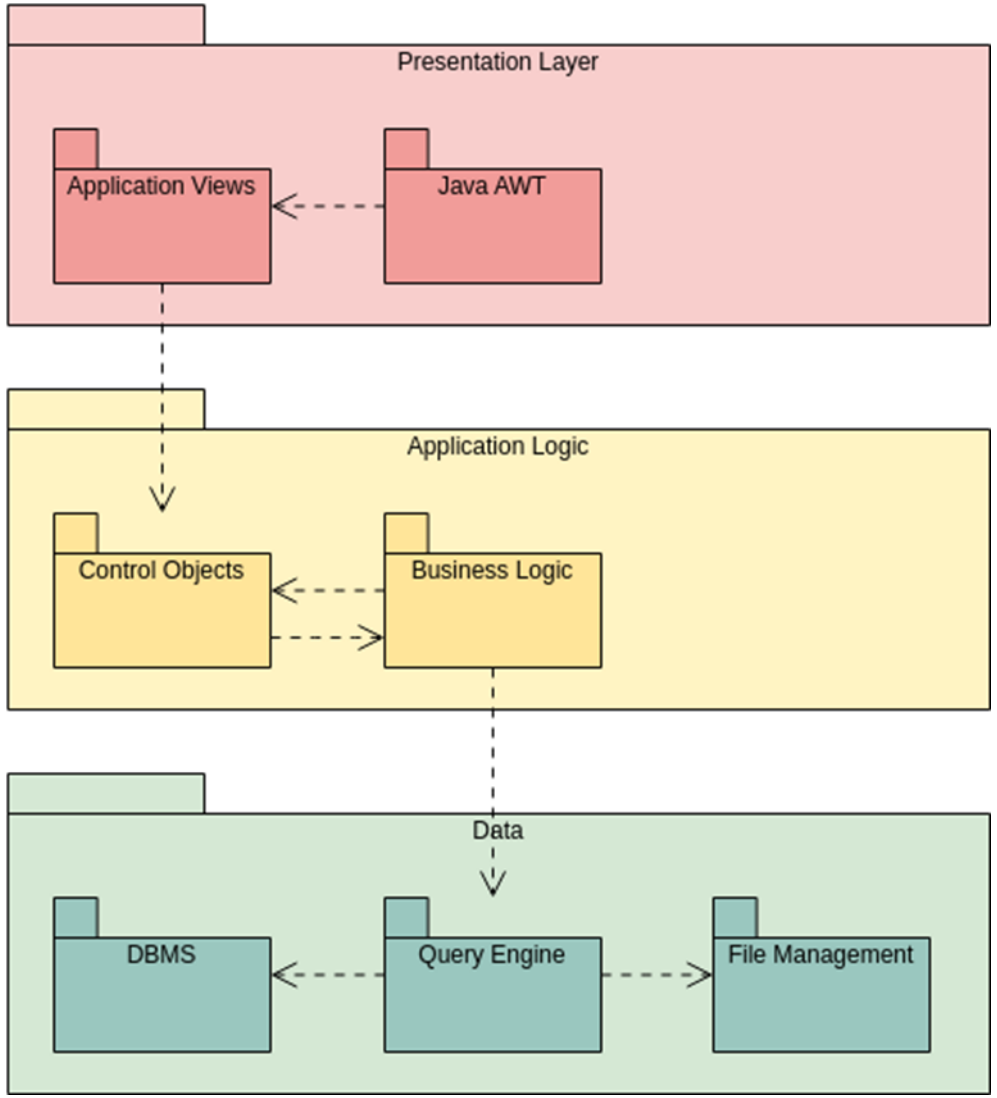




Vd Trực quan hóa các tầng

- Presentation,
- Business Logic,
- Data Management.

- Sơ đồ gói UML là cách tuyệt vời để hình dung kiến trúc logic của một hệ thống bằng
  - cách tổ chức các phần tử thành các nhóm riêng biệt và
  - thể hiện mối quan hệ phụ thuộc giữa chúng.



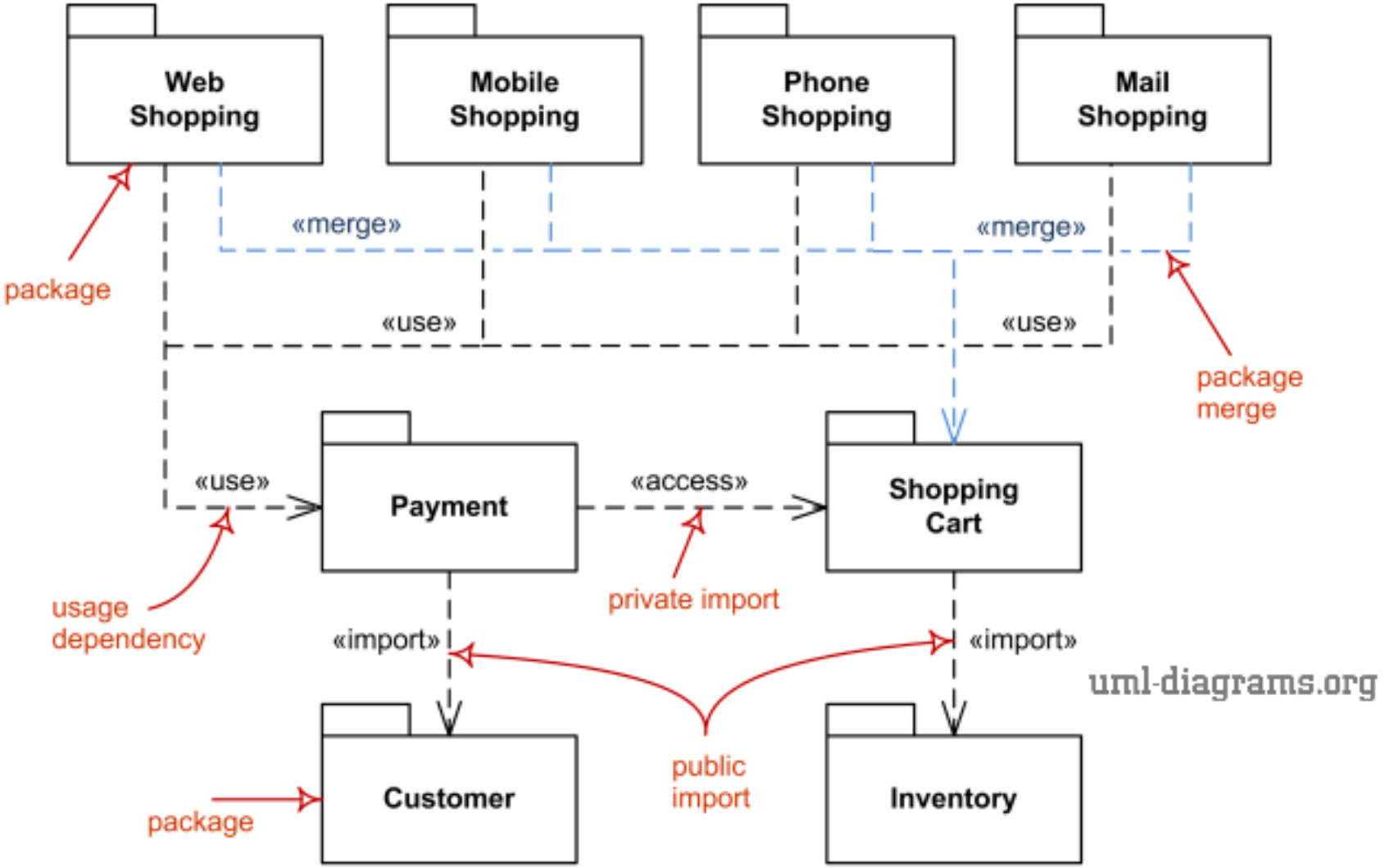
Lợi ích của sơ đồ gói

- Tổ chức thiết kế một cách logic:
  - Giúp nhóm các lớp có mối quan hệ lại với nhau.
  - Các lớp cộng tác chặt chẽ trong một use case cũng có thể được nhóm lại.
- Đơn giản hóa mô hình phức tạp:
  - giúp mô hình dễ đọc và dễ quản lý hơn ở mức độ phức tạp hợp lý.
  - Giúp phân chia một biểu diễn hệ thống quá tải thành các phân vùng (partitions) hoặc các lớp (layers).
- Quản lý Namespace:
  - gói cung cấp không gian tên (namespace) cho các thành phần mà nó chứa, đảm bảo mỗi thành phần có một tên duy nhất trong gói đó.
- Kiểm soát khả năng hiển thị (Visibility):
  - kiểm soát những thành phần nào trong gói có thể được truy cập từ bên ngoài,

Quy trình 5 bước để tạo Sơ đồ Gói:

- 1. Xác định Bối cảnh (Set the Context):
  - Quyết định mục đích của sơ đồ:
  - Bạn đang mô hình hóa các phân vùng (partitions) hay các lớp (layers) của hệ thống?.
  - Ví dụ: mô hình hóa các gói trong tầng Miền Vấn đề (Problem Domain Layer).
- 2. Nhóm các Lớp (Cluster Classes):
  - Gom các lớp có liên quan chặt chẽ vào cùng một nhóm.
  - Các lớp có thể được nhóm dựa trên các mối quan hệ như thừa kế, tập hợp, các liên kết (association) và các thông điệp gửi/nhận giữa các đối tượng.
- 3. Tạo các Gói (Create Packages):
  - Mô hình hóa mỗi nhóm lớp đã được phân cụm thành một gói (package) trên sơ đồ.
- 4. Xác định các Mối quan hệ Phụ thuộc (Identify Dependencies):
  - Xem xét các mối quan hệ (liên kết, thông điệp) giữa các lớp nằm trong các gói khác nhau để xác định sự phụ thuộc giữa các gói.
- 5. Vẽ Sơ đồ (Lay Out and Draw the Diagram):
  - Sắp xếp các gói và các mối quan hệ phụ thuộc lên sơ đồ một cách rõ ràng, dễ đọc

Ví dụ

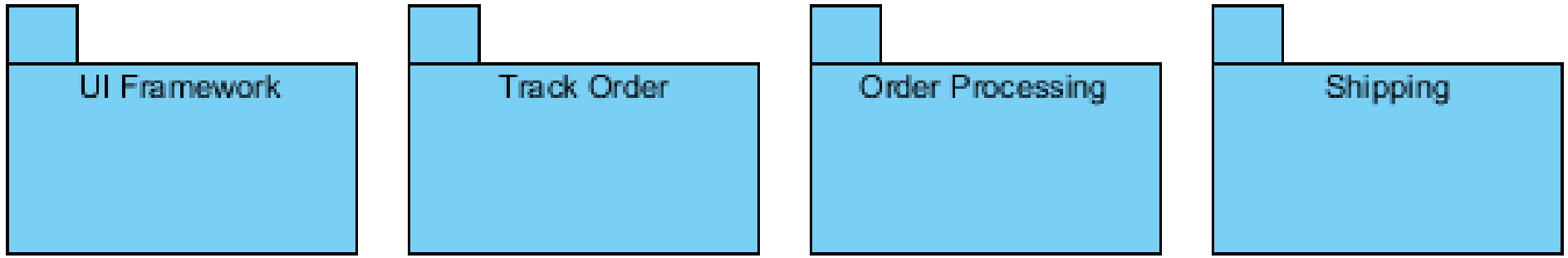

Web Shopping
Mobile Shopping
Phone Shopping
Mail Shopping
«merge»
«merge»
package
«use»
«use»
package merge
«use»
Payment
«access»
Shopping Cart
usage dependency
private import
«import»
«import»
uml-diagrams.org
public import
Customer
Inventory
package

Bài tập

- Hệ thống xử lý đơn hàng - Mô tả vấn đề
  - kịch bản " Track Order " cho một cửa hàng mua sắm trực tuyến.
  - Mô-đun Track Order
    - chịu trách nhiệm cung cấp thông tin theo dõi cho các sản phẩm khách hàng đã đặt.
    - Khách hàng nhập số sê-ri theo dõi,
    - mô-đun Track Order sẽ tham chiếu hệ thống và cập nhật trạng thái giao hàng hiện tại cho khách hàng.
- Tổ chức các lớp trong hệ thống thành các gói (package) theo nguyên tắc:
  - Cố kết mạnh: liên kết giữa các lớp trong cùng một gói mạnh
  - Tương liên yếu: liên kết giữa các lớp thuộc các gói khác nhau yếu

Xác định các gói của hệ thống

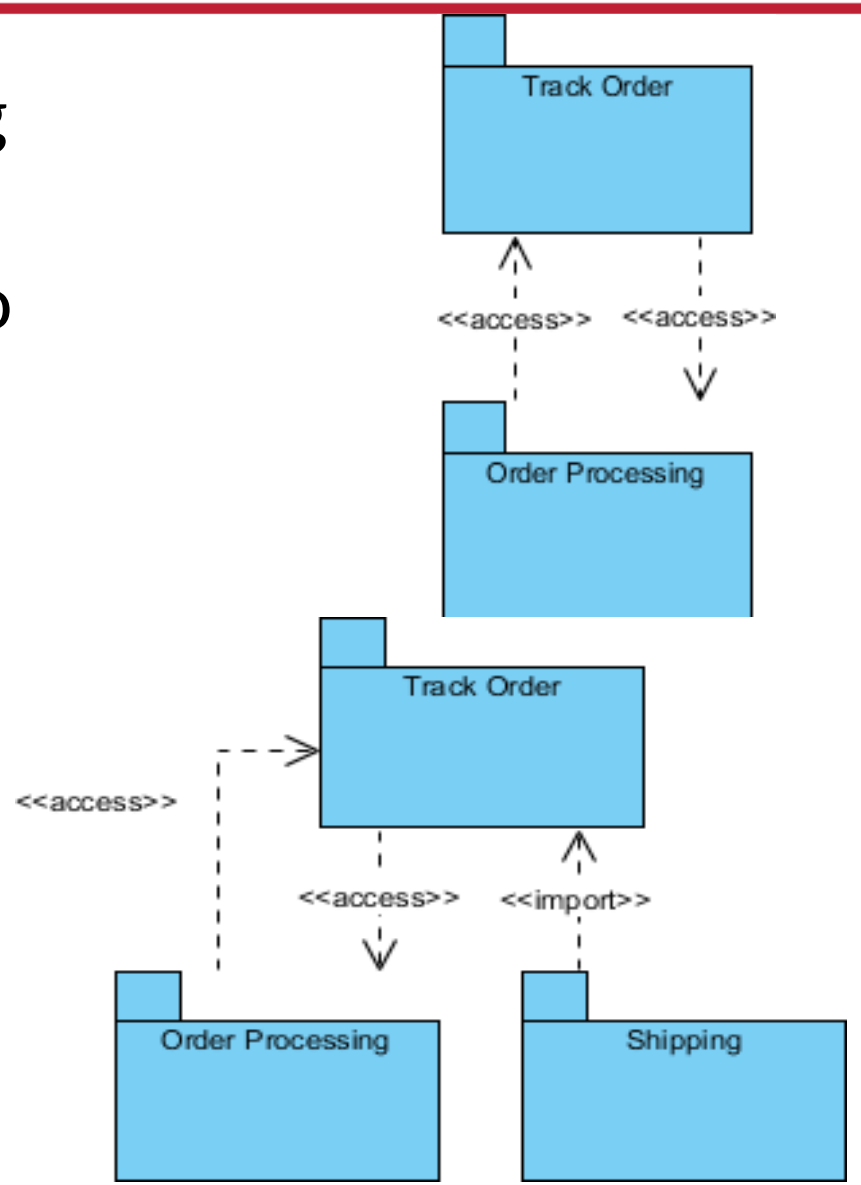
- Có một mô-đun theo dõi đơn hàng,
  - mô-đun này phải trao đổi với các mô-đun khác để biết thông tin chi tiết về đơn hàng, chúng ta hãy gọi nó là " Order Details"".
- Tiếp theo sau khi lấy Chi tiết đơn hàng,
  - nó phải biết thông tin về vận chuyển, chúng ta hãy gọi đó là " "Shipping ".



Xác định các phụ thuộc trong Hệ thống

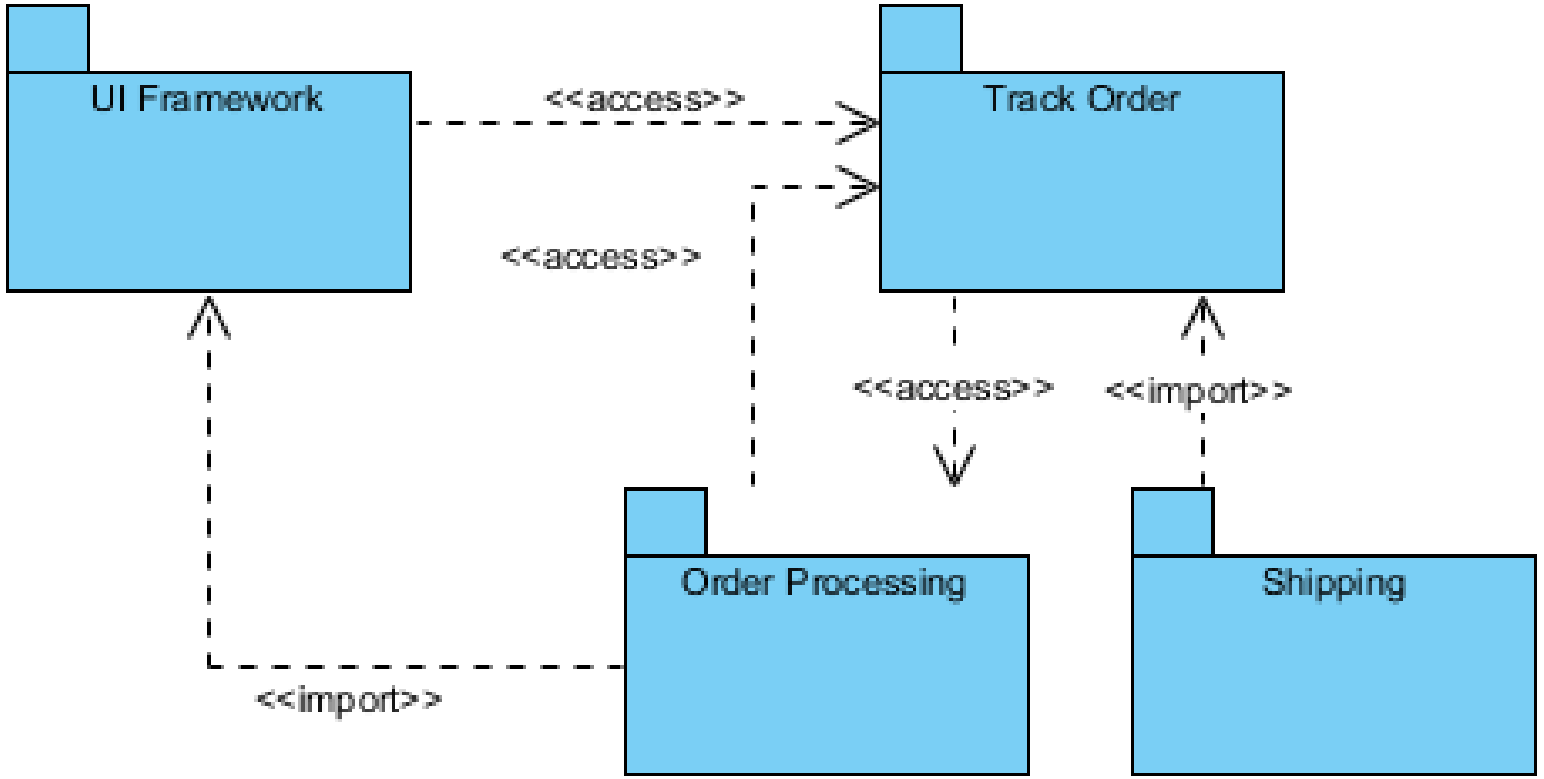
- Track order cần lấy thông tin chi tiết đơn hàng từ "Order Details" và
- "Order Details" phải biết thông tin theo dõi do khách hàng cung cấp.
- Hai mô-đun đang truy cập lẫn nhau, điều này đủ để tạo ra sự phụ thuộc kép <<truy cập>>

- Để biết thông tin vận chuyển, "Shipping" có thể nhập "Track order " để điều hướng dễ dàng hơn.



Sơ đồ gói cho hệ thống con Xử lý đơn hàng.

- Cuối cùng, sự phụ thuộc của Track Order vào UI Framework cũng được ánh xạ để hoàn thiện



Component Diagram

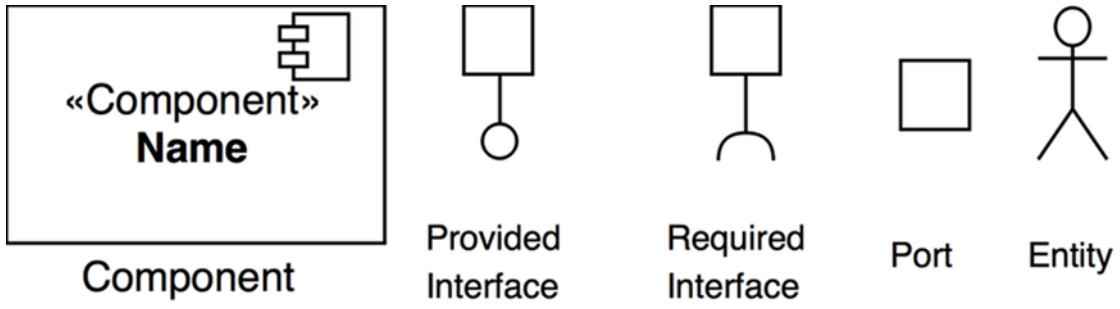
Sơ đồ Thành phần (Component Diagram)

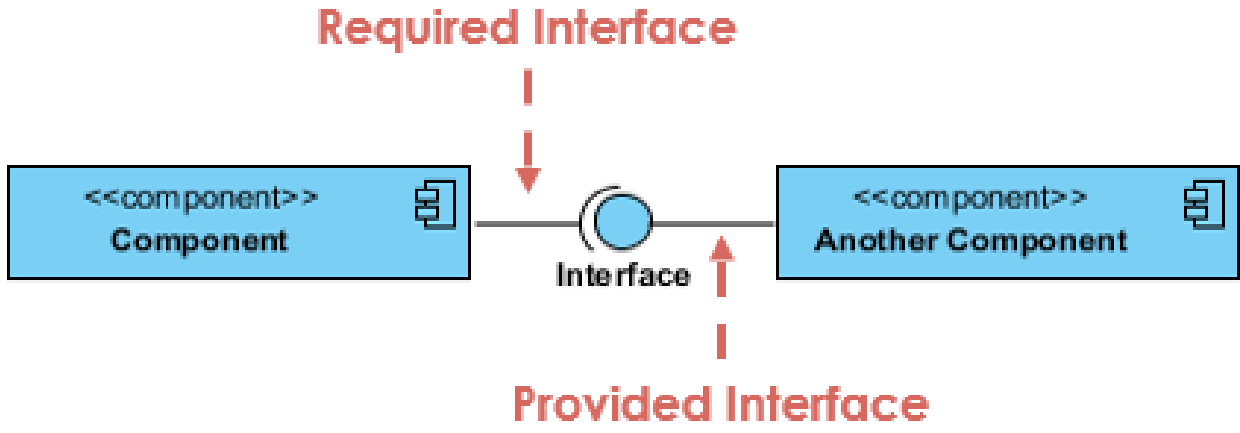
- Component là gì?
  - Là một module thực thi hoặc chương trình (ví dụ: .exe, .dll, .jar).
  - Chúng là các module vật lý, có thể tái sử dụng và có các giao diện rõ ràng để kết nối với nhau.
  - ""
- Component Diagram:
  - Là một sơ đồ cấu trúc của UML.
  - chia nhỏ hệ thống thực tế đăng phát triển thành nhiều cấp độ chức năng cao khác nhau.
  - Mỗi thành phần chịu trách nhiệm cho một mục tiêu rõ ràng trong toàn bộ hệ thống và chỉ tương tác với các thành phần thiết yếu khác khi cần thiết.
  - Mô tả khía cạnh tĩnh của việc triển khai (static implementation view).

- *Hãy nghĩ về các thành phần (components) như những 'viên gạch Lego' của phần mềm.*
- *Sơ đồ Thành phần giúp chúng ta thấy cách các 'viên gạch' này lắp ráp thành một hệ thống*

Ký hiệu trong Sơ đồ Thành phần

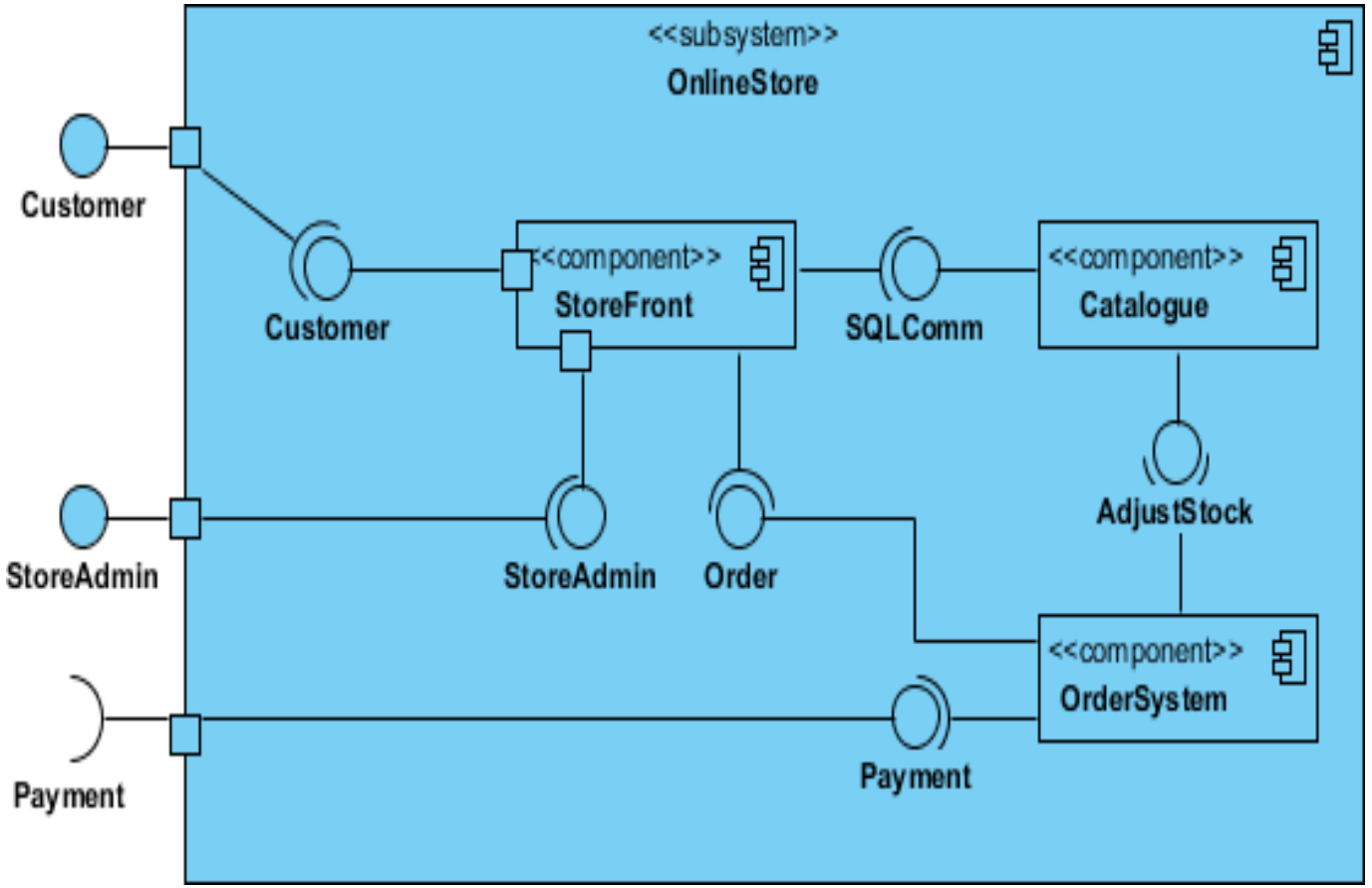
- Component:
  - Hình chữ nhật với biểu tượng nhỏ ở góc trên bên phải.
- Interface (Giao diện): Cách component tương tác.
  - Provided Interface (Giao diện được cung cấp):
  - Required Interface (Giao diện yêu cầu):
- Port (Cổng):
  - Là điểm kết nối để component tương tác với môi trường bên ngoài.





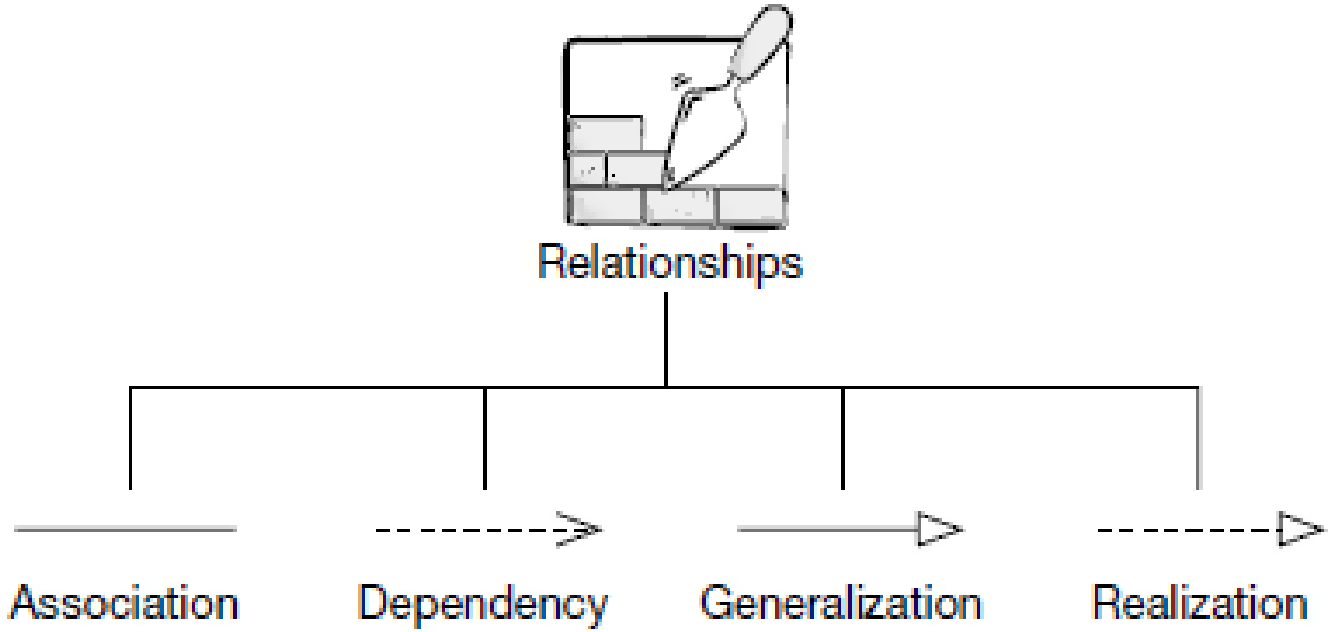
Hệ thống con

- Bộ phân loại hệ thống con là một phiên bản chuyên biệt của bộ phân loại thành phần.
- Do đó, phần tử ký hiệu hệ thống con kế thừa tất cả các quy tắc giống như phần tử ký hiệu thành phần.
- Điểm khác biệt hần tử ký hiệu "subsystem" thay vì "component".



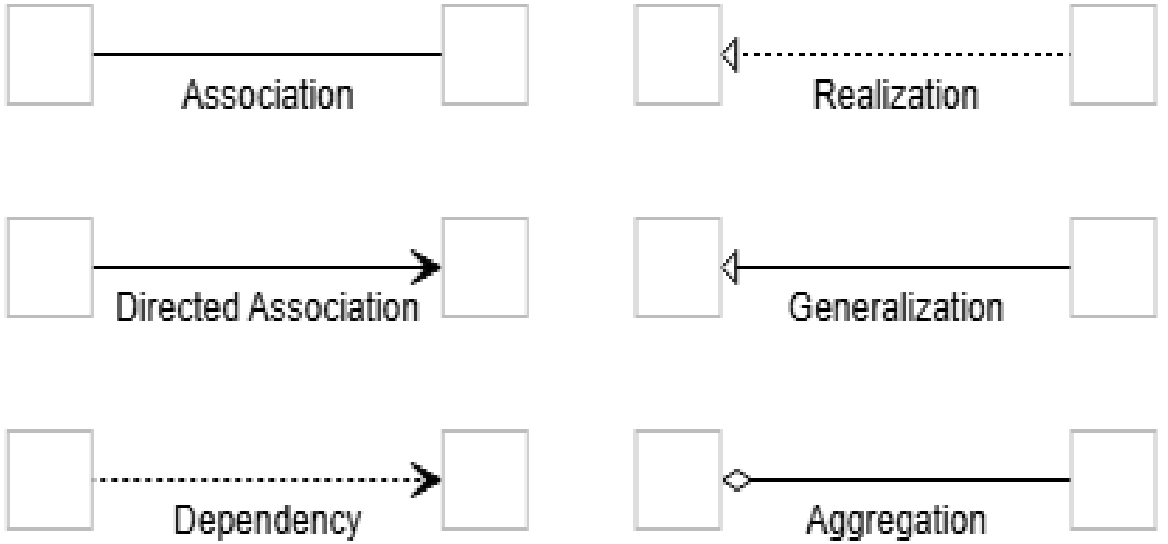
"Mối quan hệ Phụ thuộc (Dependency Relationship)

- Dependency (Phụ thuộc):
  - Cho thấy một thành phần (client) phụ thuộc vào một thành phần khác (supplier).
  - Nếu thành phần supplier thay đổi, thành phần client có thể bị ảnh hưởng.
  - Ký hiệu: Mũi tên nét đứt, cho thấy sự thay đổi ở một component có thể ảnh hưởng đến component khác.



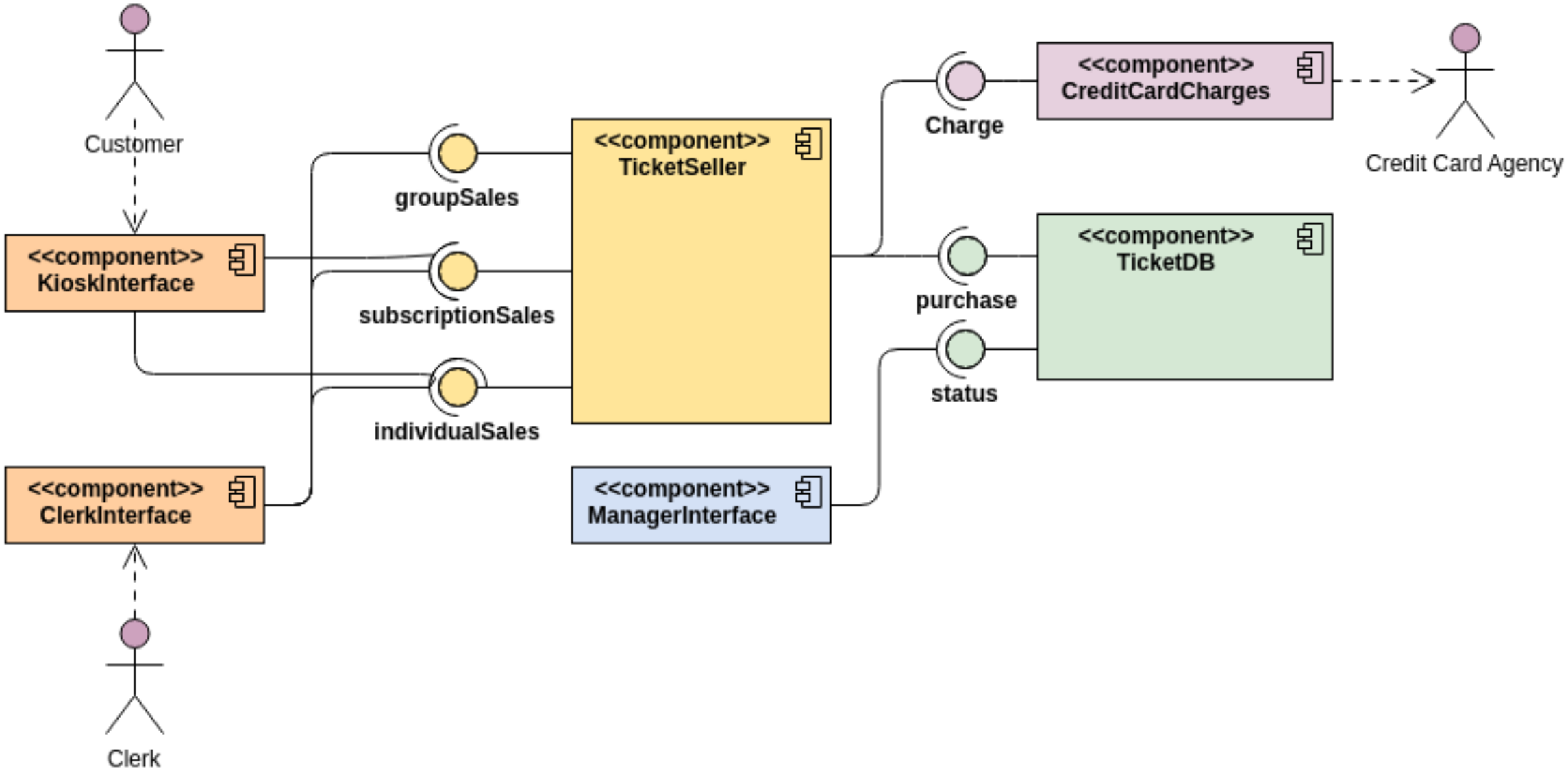
Mối quan hệ khác

- **Association**: :
  - Một liên kết chỉ định mối quan hệ ngữ nghĩa có thể xảy ra giữa các trường hợp
- Realization (Hiện thực hóa):
  - Mối quan hệ giữa một component và các lớp (classes) hiện thực hóa nó.
- **Aggregation**
  - loại liên kết có một đầu được đánh dấu là chung như một loại tổng hợp



Biểu đồ thành phần (1)

- 5 khuôn dập chuẩn của UML
  - **<<executable>>:** Một thành phần có thể thực hiện trên 1 nút
  - **<<library>>:** Một thư viện đối tượng tĩnh hoặc động
  - **<<table>>:** Một bảng trong cơ sở dữ liệu
  - **<<file>>:** Một tập tin chứa mã nguồn hoặc dữ liệu
  - **<<document>>:** Một tài liệu
- Có thể dùng các khuôn dập không chuẩn
  - **<<application>>:** Một ứng dụng
  - **<<database>>:** Một cơ sở dữ liệu
  - **<<infrastructure>>:** Một thành phần cơ sở hạ tầng (vd: một dịch vụ lưu trữ, một kiểm soát đăng nhập,...)
  - **<<source code>>:** Một tập tin mã nguồn
  - **<<web service>>:** Một dịch vụ Web


Customer
<<component>>
KioskInterface
<<component>>
ClerkInterface
Clerk
groupSales
subscriptionSales
individualSales
<<component>>
TicketSeller
<<component>>
ManagerInterface
Charge
purchase
status
<<component>>
CreditCardCharges
<<component>>
TicketDB
Credit Card Agency

Phân loại Component trong UML:

- Deployment components:
  - Các thành phần cần thiết để tạo thành một hệ thống có thể thực thi được,
  - ví dụ: file thực thi (EXE) và thư viện (DLL).
- Work product components:
  - Các sản phẩm của quá trình phát triển, bao gồm các tệp mã nguồn, các tệp dữ liệu, từ đó mà ta đã tạo lập ra các thành phần triển khai
  - Các thành phần này không trực tiếp tham gia vào hệ thống thực thi, nhưng không có chúng thì không tạo được hệ thống thực thi
  - ví dụ: file mã nguồn và file dữ liệu, dùng để tạo ra các deployment component.
- Execution components:
  - Các thành phần được tạo ra khi hệ thống đang chạy, Đó là các thành phần được tạo nên như là một kết quả của một hệ thực hiện
  - ví dụ như một đối tượng COM+ được khởi tạo từ một DLL

Ứng dụng Sơ đồ Thành phần phổ biến

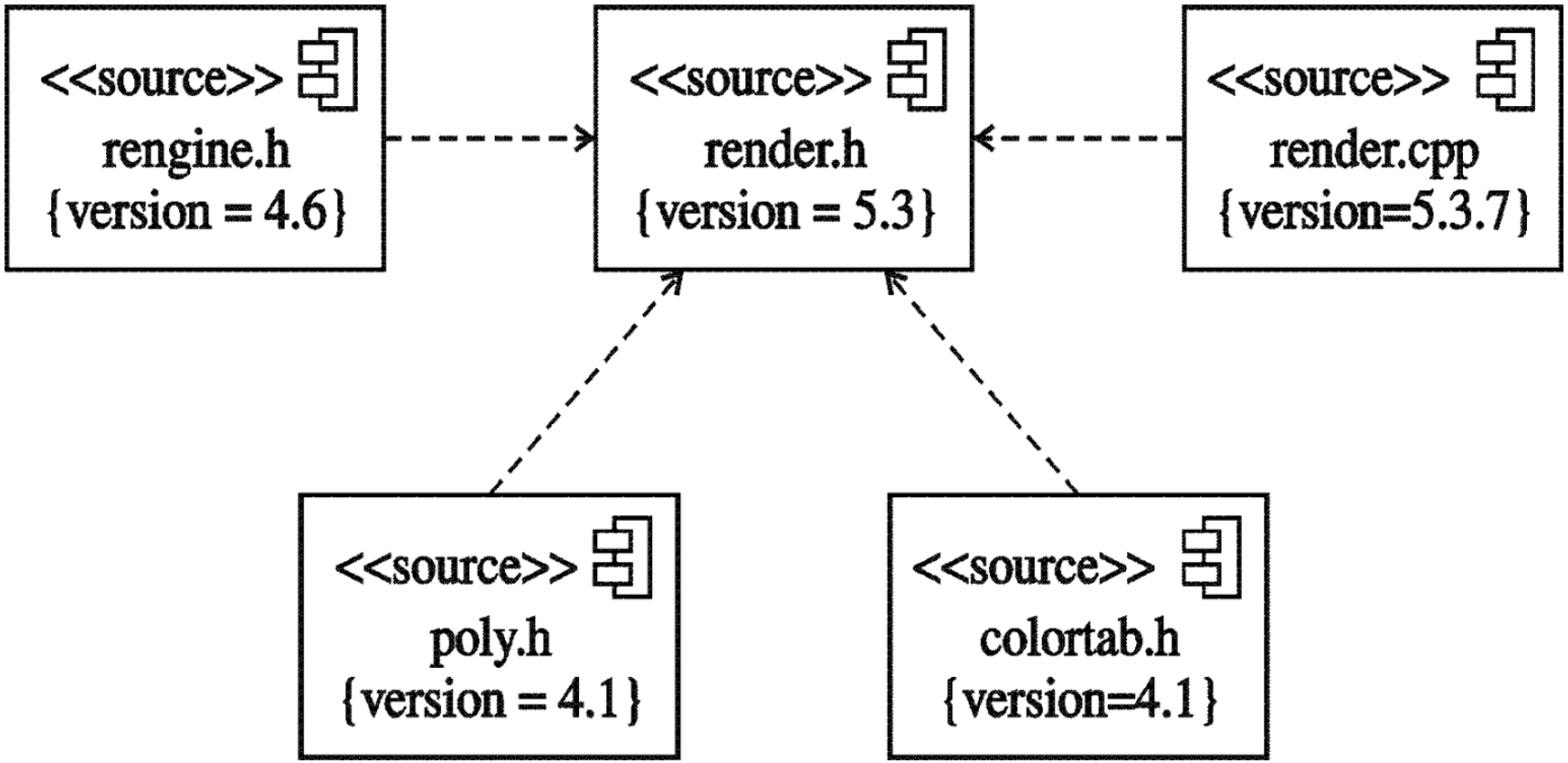
- 1. Mô hình hóa Mã nguồn (Source Code):
  - Các tệp mã nguồn hoặc mô-đun được biểu diễn dưới dạng các thành phần
- 2. Mô hình hóa các Bản phát hành Thực thi (Executable Releases):
  - ứng dụng, thư viện hoặc dịch vụ, được mô hình hóa như một thành phần..
- 3. Mô hình hóa Cơ sở dữ liệu Vật lý (Physical Databases):
  - Cơ sở dữ liệu vật lý, đại diện cho việc lưu trữ dữ liệu thực tế, được mô hình hóa như một thành phần.
  - Thành phần này thường bị coi là khuôn mẫu <<database>>.
- 4. Mô hình hóa các Hệ thống Thích ứng (Adaptable Systems):
  - hệ thống có khả năng thích ứng: thay đổi cấu hình hoặc sửa đổi thời gian chạy,
  - được mô hình hóa bằng các thành phần nhấn mạnh vào tính linh hoạt của chúng

1. Mô hình hoá mã nguồn (1)

- Mô hình hoá mã nguồn cũng là một mục đích của việc tạo lập biểu đồ thành phần
  - Các tệp mã nguồn dùng để chứa các chi tiết về các lớp, các giao diện, các hợp tác và các phần tử logic khác
  - Các tệp mã nguồn tạo nên một bước trung gian để tạo lập các thành phần vật lý/thực thi
- Các công cụ (vd: chương trình biên dịch) thường đòi hỏi các tệp mã nguồn phải được tổ chức theo một quy cách nhất định nào đó
  - (vd: tệp header .h và tệp mã nguồn .cpp).
  - Các mối liên quan phụ thuộc giữa các tệp này phản ánh một trật tự biên dịch

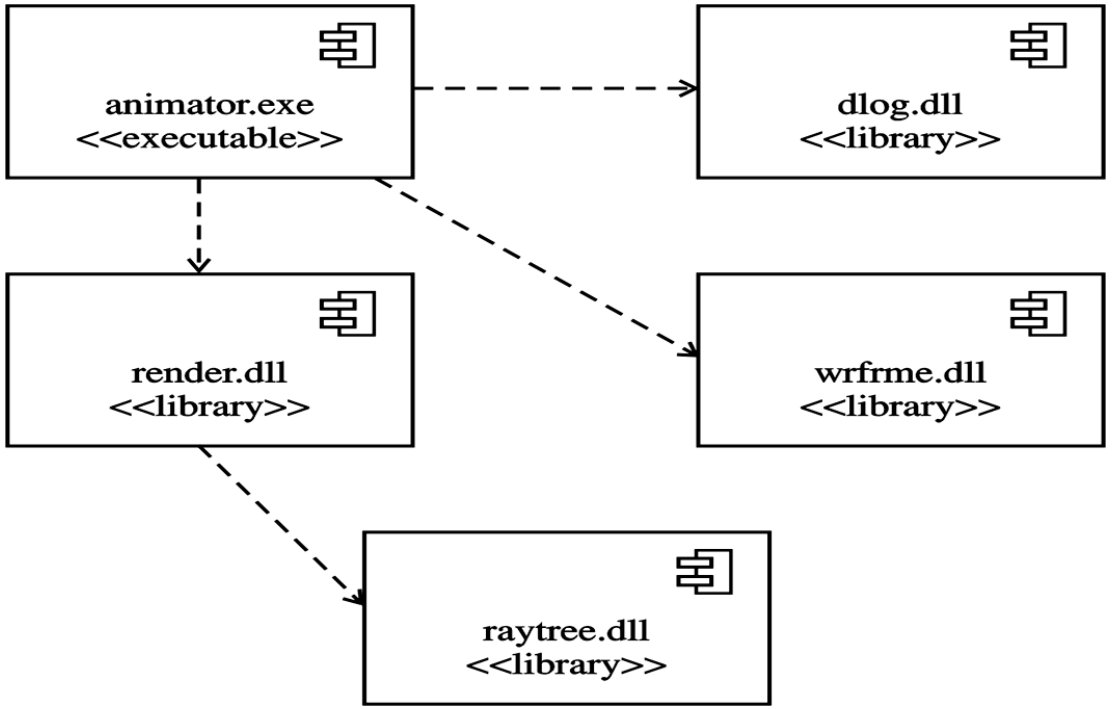
Mô hình hoá mã nguồn (2)

- Ví dụ: Mô hình hoá các tệp mã nguồn dùng để xây dựng thư viện render.dll từ ví dụ trước



2.Mô hình hoá các deployment component và thư viện

- Mục đích chính của việc sử dụng Biểu đồ thành phần
  - là để mô hình hoá các thành phần triển khai, tạo nên sự cài đặt của hệ thống
- Nếu cài đặt một hệ thống chỉ úng một tệp (.EXE),
  - thì chẳng cần dùng tới thành phần
- Nhưng nếu hệ thống gồm nhiều tệp khả thi và liên kết với các thư viện đối tượng,
  - thì cần dùng biểu đồ thành phần để giúp hiển thị, đặc tả, thành lập và tư liệu hoá các quyết định đối với hệ thống vật lý

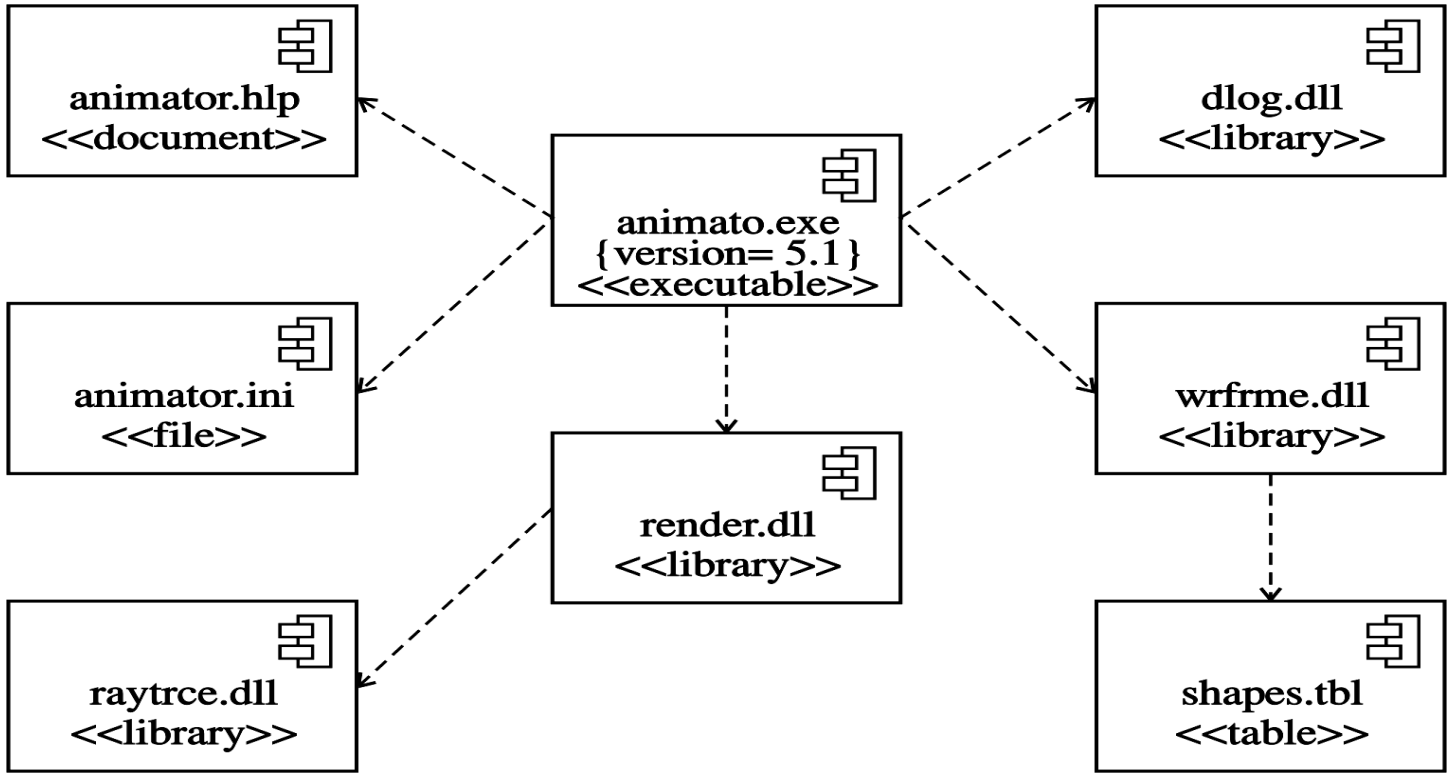


Mô hình hoá các bảng, các tệp và các tài liệu (1)

- Bên cạnh các tệp khả thi và thư viện tạo nên phần chạy được của hệ thống,
  - thì còn nhiều thành phần bố trí khác, gọi là các thành phần phụ trợ, cần cho việc cài đặt hệ thống
- Ví dụ:
  - Để cài đặt, ta vẫn cần các tệp dữ liệu, các tài liệu trợ giúp, các scripts, các tệp log, các tệp khởi tạo tham số cấu hình,...
- Mô hình hoá các thành phần này cũng là một phần quan trọng để diễn tả hình trạng của hệ thống

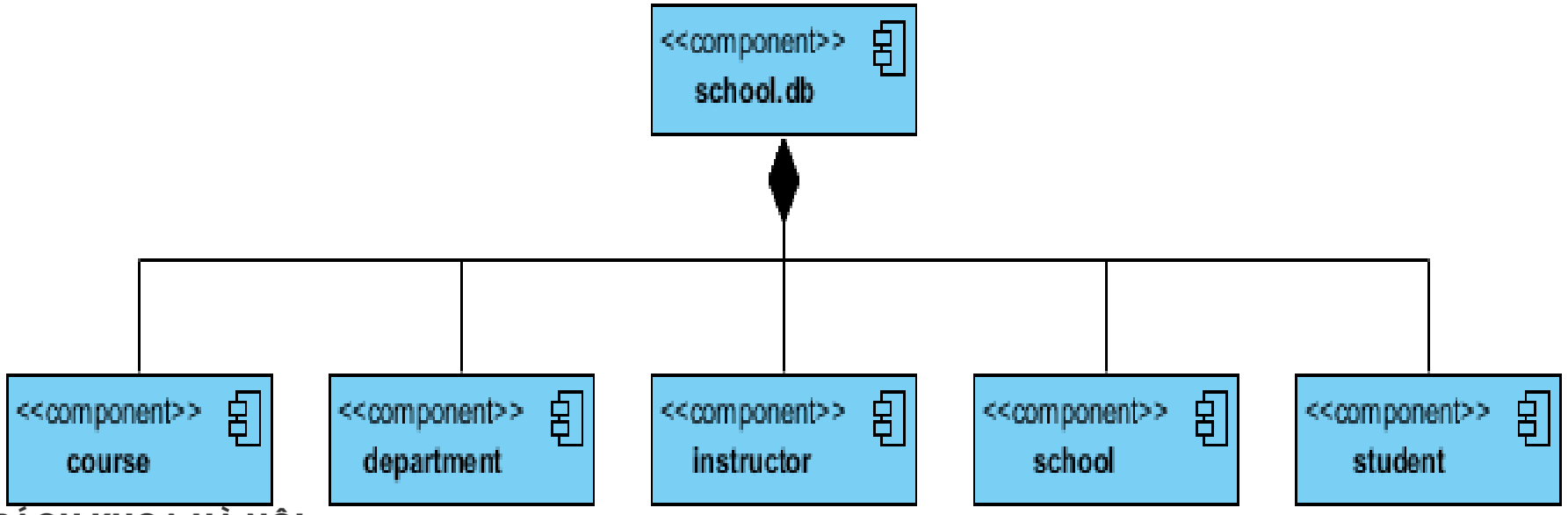
Mô hình hoá các bảng, các tệp và các tài liệu (2)

- Ví dụ: Từ biểu đồ, thêm vào các bảng dữ liệu, các tệp và các tài liệu



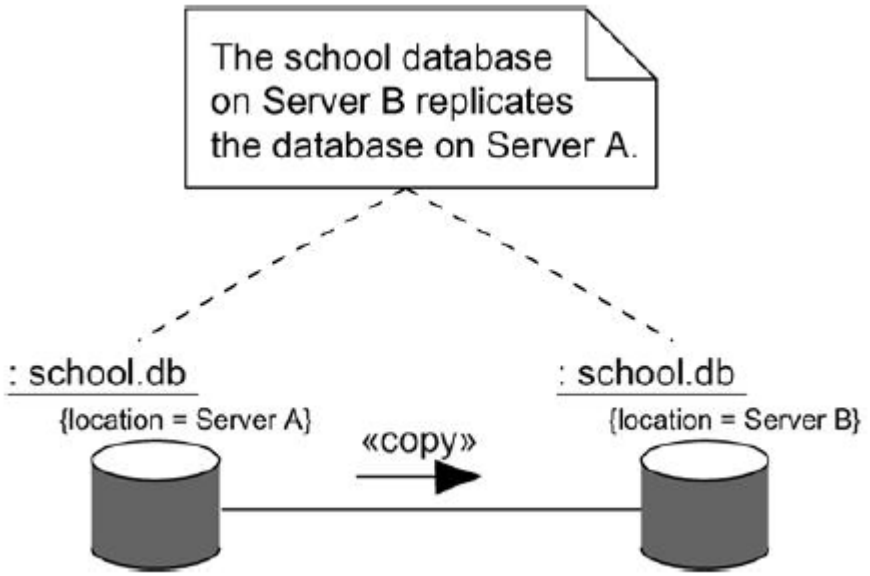
3.Mô hình hóa cơ sở dữ liệu vật lý

- Xác định các lớp trong mô hình đại diện cho lược đồ cơ sở dữ liệu logic
- Chọn chiến lược để ánh xạ các lớp này vào các bảng.
- Chiến lược ánh xạ sẽ bị ảnh hưởng bởi vị trí dữ liệu lưu trữ trên hệ thống triển khai.
- tạo sơ đồ chứa các thành phần được mô phỏng theo dạng bảng.
- chuyển đổi thiết kế logic thành thiết kế vật lý.



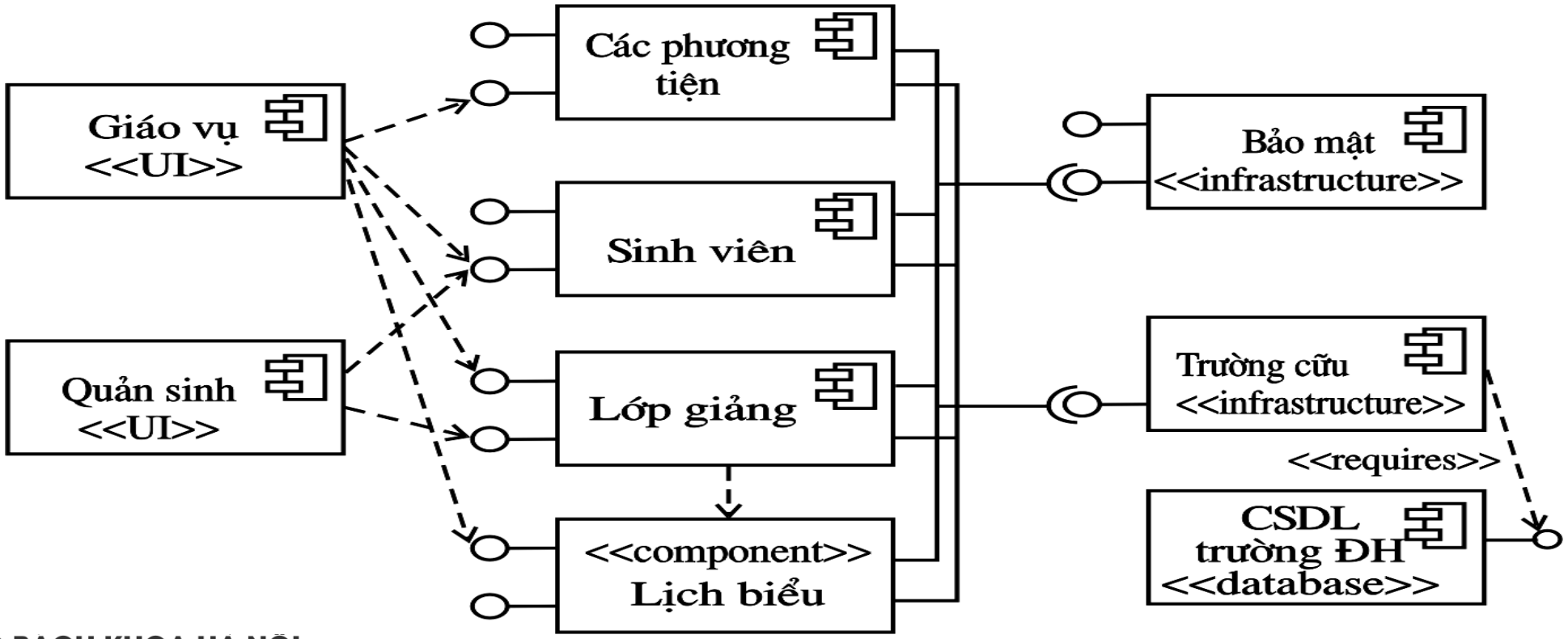
4. Mô hình hóa các hệ thống thích ứng

- Hãy xem xét sự phân bố vật lý của các thành phần có thể di chuyển từ nút này sang nút khác.
- chỉ định vị trí của một thể hiện thành phần
  - bằng cách đánh dấu nó bằng một giá trị được gắn thẻ vị trí.
- Nếu bạn muốn mô hình hóa các hành động khiến một thành phần di chuyển,
  - hãy tạo một sơ đồ tương tác tương ứng chứa các thể hiện của thành phần.
  - minh họa sự thay đổi vị trí bằng cách vẽ cùng một thể hiện nhiều lần,
    - nhưng với các giá trị khác nhau cho giá trị được gắn thẻ vị trí của nó.



Ví dụ Biểu đồ thành phần (2)

- Để tổ chức các thành phần lại với nhau, ta có hai cách:
  - Gom các thành phần vào các gói, hoặc
  - Thiết lập các mối liên quan phụ thuộc giữa chúng,



(Deployment Diagram)

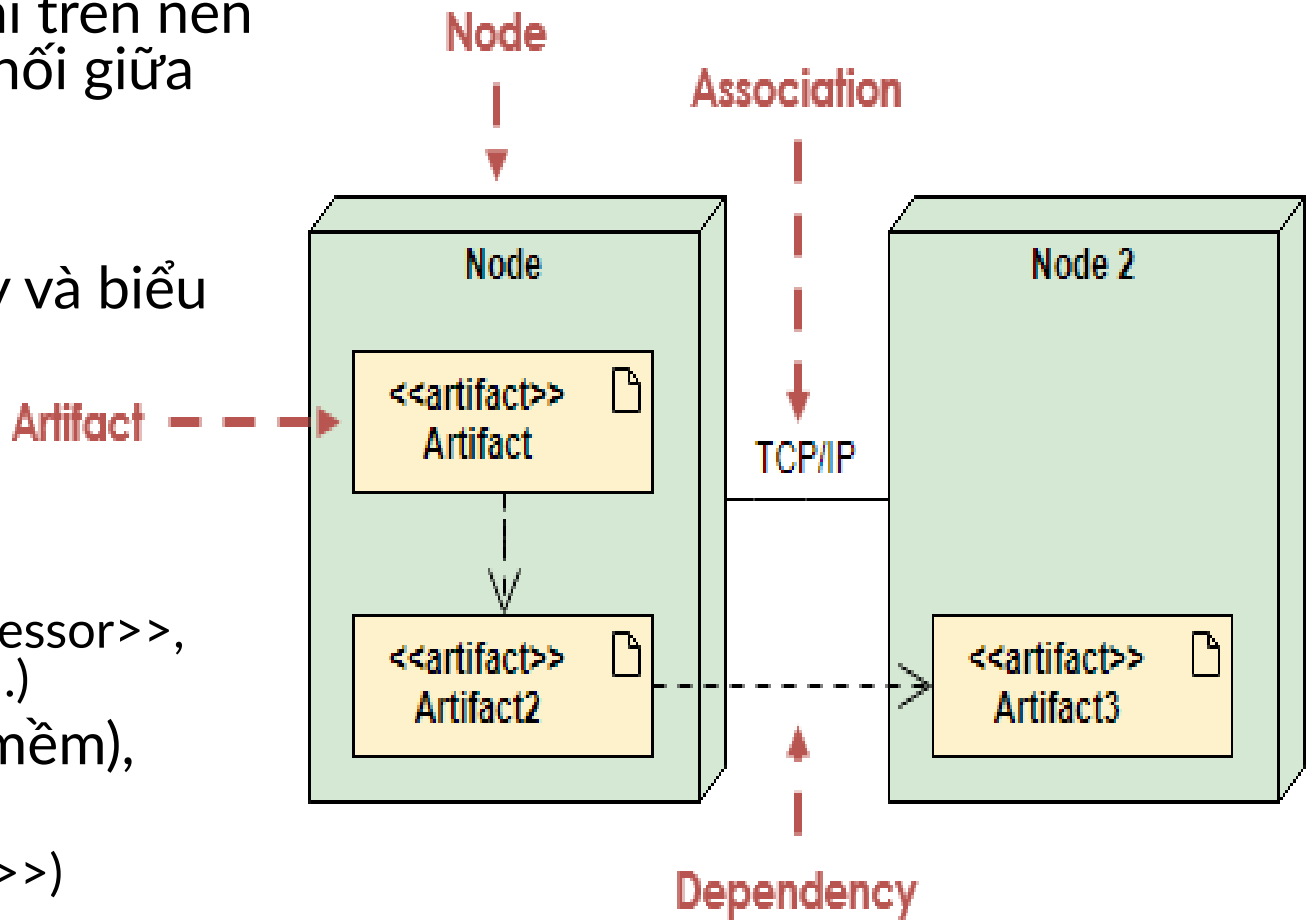
https://www.youtube.com/watch?v=IzEzX5HW_CU

Sơ đồ Triển khai (Deployment Diagram)

- Deployment Diagram:
  - sơ đồ kiến trúc phần mềm mô hình hóa việc triển khai vật lý các hiện vật (artifacts) phần mềm lên các nút phần cứng. (processing nodes)
  - nó cung cấp chế độ xem tĩnh (static deployment view), cấp cao về kiến trúc thực thi của hệ thống,
    - hiển thị nơi các thành phần phần mềm sẽ chạy và cách các bộ phận khác nhau được kết nối vật lý như thế nào.
- Đây là sơ đồ mô tả kiến trúc vật lý của hệ thống."
  - "Nếu Sơ đồ Thành phần mô tả 'cái gì' (các module phần mềm),
    - thì Sơ đồ Triển khai mô tả 'ở đâu' (được cài đặt và chạy trên phần cứng nào).

executable artifacts và các nút phần cứng

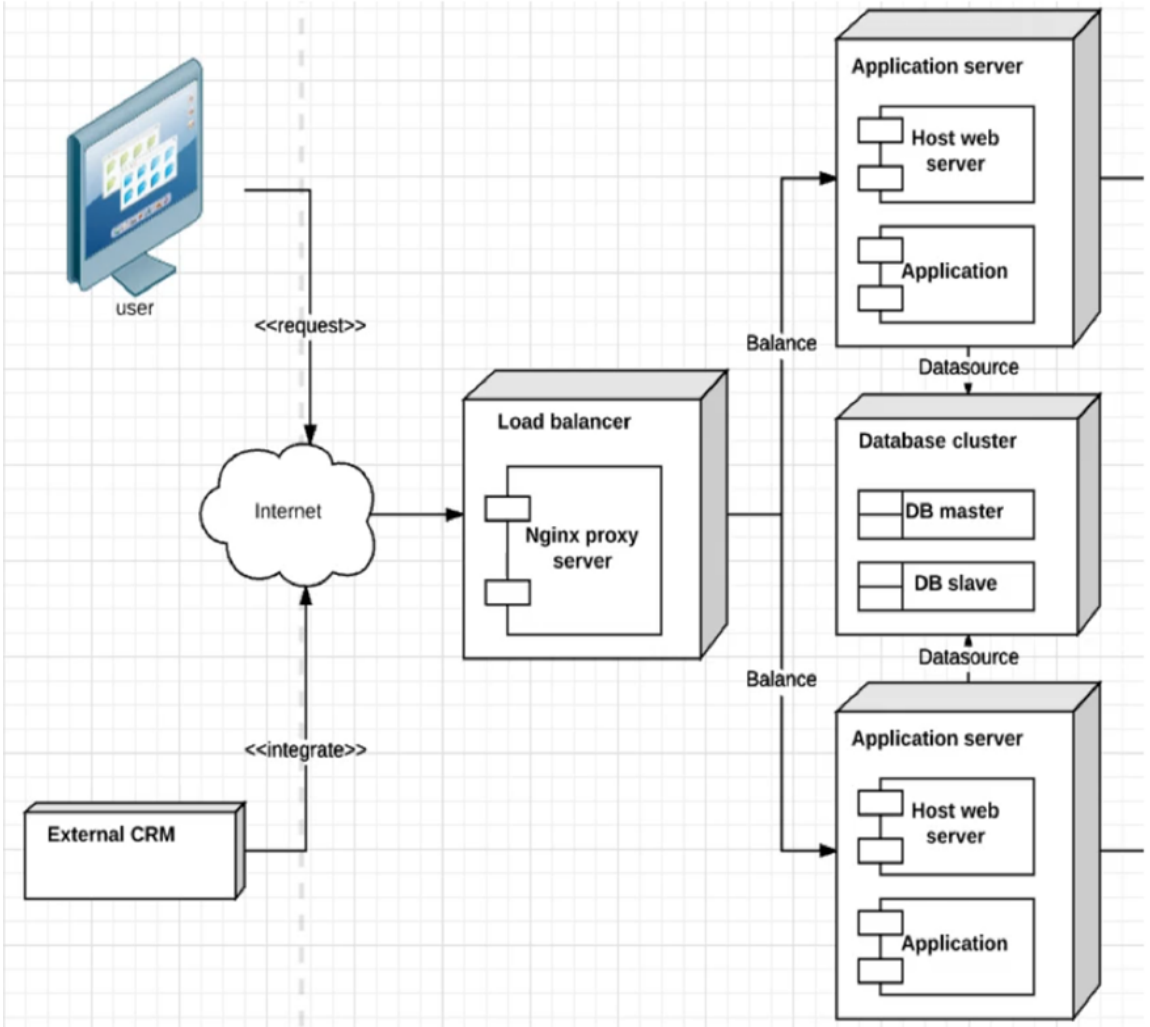
- Biểu đồ triển khai (Deployment diagram)
  - diễn tả sự bố trí các đối tượng thực thi trên nền tảng platform) qua các nút và các kết nối giữa các nút đó
- Một nút (node) là
  - một phần tử vật lý tồn tại vào lúc chạy và biểu diễn cho một tài nguyên tính toán (computational resource).
- Nút có thể là:
  - Một thiết bị (nút phần cứng),
    - <<device>> (hoặc cụ thể hơn là <<processor>>, <<console>>, <<kiosk>>, <<printer>>,...)
  - Một môi trường thực hiện (nút phần mềm),
    - <<execution env>>
    - (vd: <<EJB Container>>, <<J2EEServer>>)

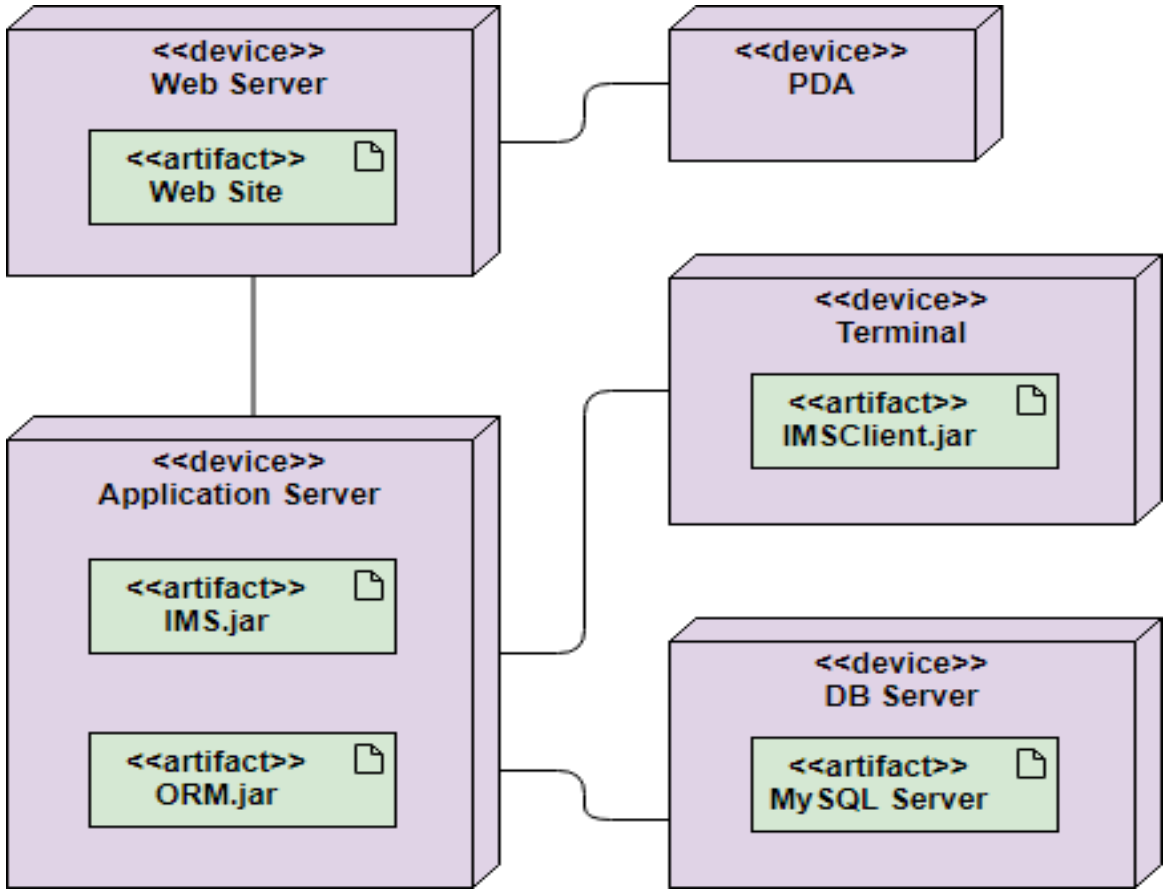


Ký hiệu trong Sơ đồ Triển khai

- "Các ký hiệu chính trong Sơ đồ Triển khai giúp chúng ta biểu diễn các yếu tố phần cứng và cách phần mềm được phân bổ trên đó."
  1. Node (Nút): Yếu tố vật lý có khả năng xử lý
  2. Artifact (Hiện vật): Một phần vật lý của hệ thống phần mềm
     - (ví dụ: file .exe, .war, database).
     - Ký hiệu: Hình chữ nhật với stereotype <<artifact>>.
  3. Communication Path (Đường truyền thông): Kết nối giữa các nút
     - (ví dụ: LAN, WAN, Internet). Ký hiệu: Đường thẳng nối giữa các nút.
  4. Deployment Relationship (Quan hệ Triển khai):
     - Cho thấy artifact được triển khai trên node.
     - Ký hiệu: Mũi tên nét đứt với stereotype <<deploy>>, hoặc artifact được vẽ lồng bên trong node.

Ví dụ Deployment Diagram



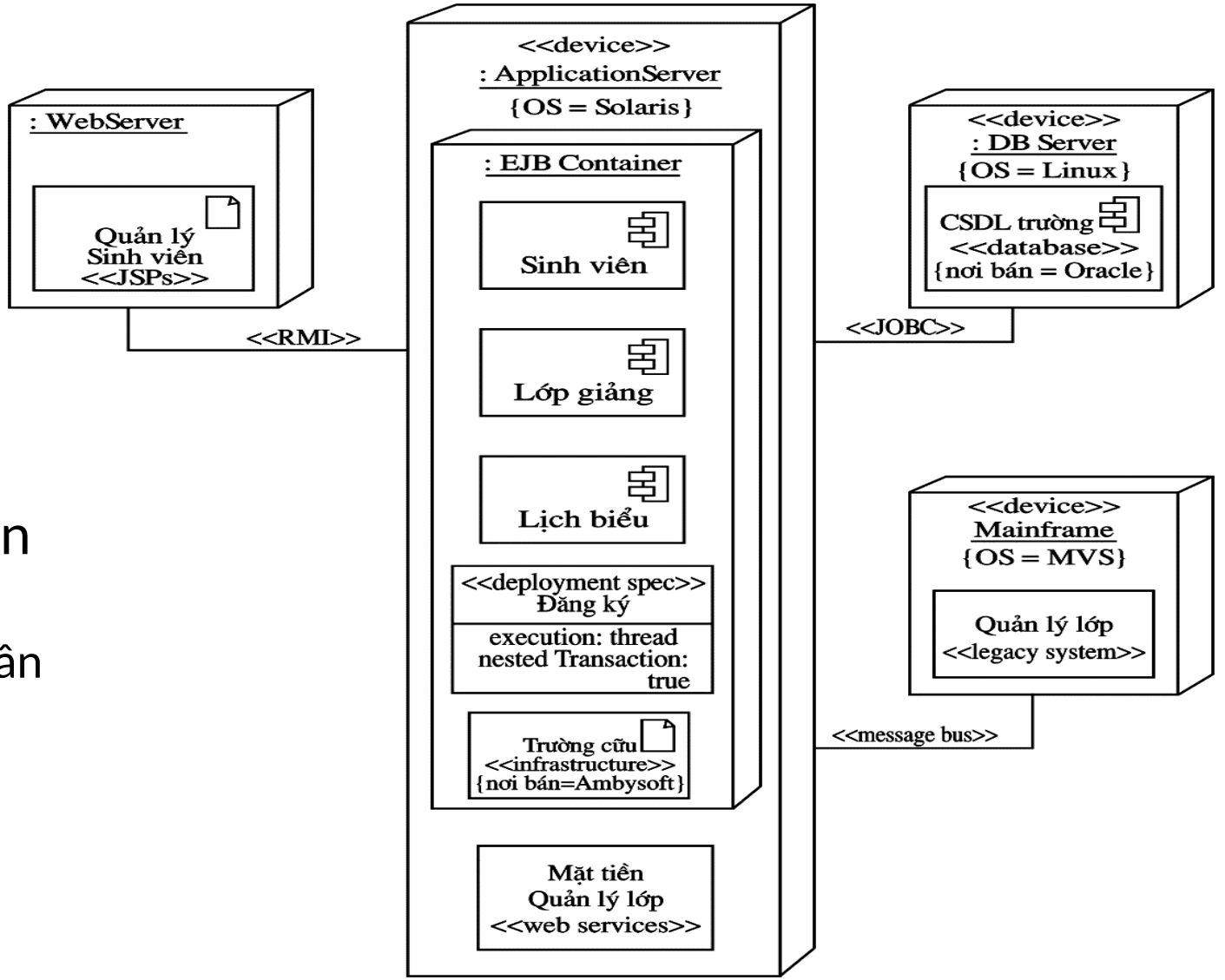


Các bước để tạo Biểu đồ triển khai

- Bước 1: Xác định các thành phần:
  - Liệt kê tất cả các bộ phận phần mềm và thiết bị phần cứng sẽ có trong sơ đồ triển khai.
- Bước 2: Hiểu các mối quan hệ :
  - Tìm hiểu cách các bộ phận này kết nối và hoạt động cùng nhau.
- Bước 3: Thu thập các yêu cầu:
  - Thu thập thông tin về phần cứng, thiết lập mạng và bất kỳ quy tắc đặc biệt nào cho việc triển khai.
- Bước 4: Vẽ các nút và thành phần:
  - vẽ các thiết bị phần cứng (nút) và các bộ phận phần mềm (thành phần) sử dụng các ký hiệu chuẩn,.
- Bước 5: Kết nối các nút và thành phần:
  - Sử dụng các đường thẳng hoặc mũi tên để hiển thị cách các nút và thành phần được liên kết.
- Bước 6: Thêm chi tiết:
  - Ghi nhãn bao gồm thông tin bổ sung như thông số kỹ thuật phần cứng hoặc giao thức truyền thông.
- Bước 7: Ghi chép:
  - Ghi lại mọi quyết định hoặc giả định quan trọng được đưa ra khi tạo sơ đồ.

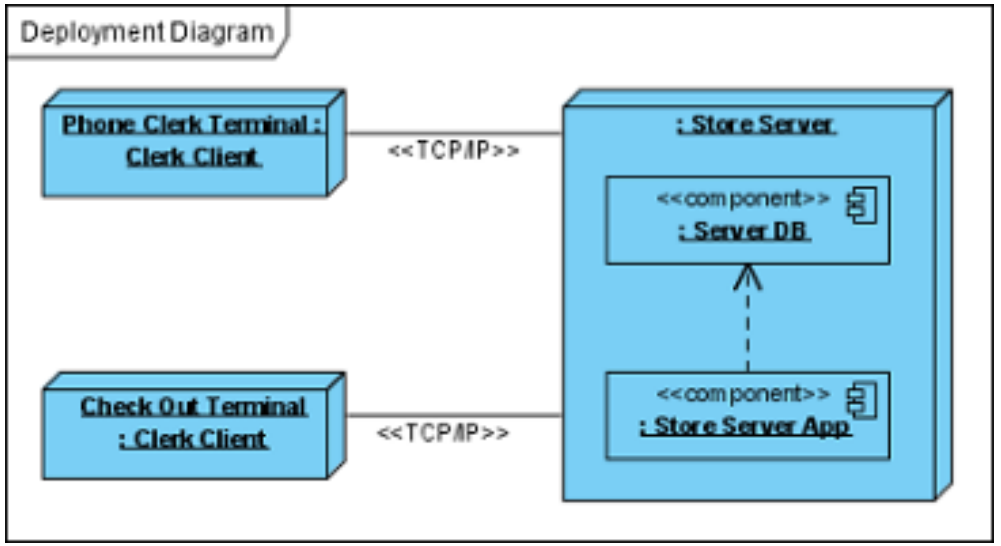
Bố trí các thành phần khả thi vào các nút

- Hệ thống thông tin của 1 trường đại học
- Nút WebServer
  - Nút này có thể là một loại phần mềm hoặc là một thiết bị vật lý
- nút ApplicationServer
  - chứa nút EJBContainer (nút phần mềm),
    - nút này lại chứa ba thành phần phần mềm,
    - một đặc tả bố trí, và
    - một phần mềm



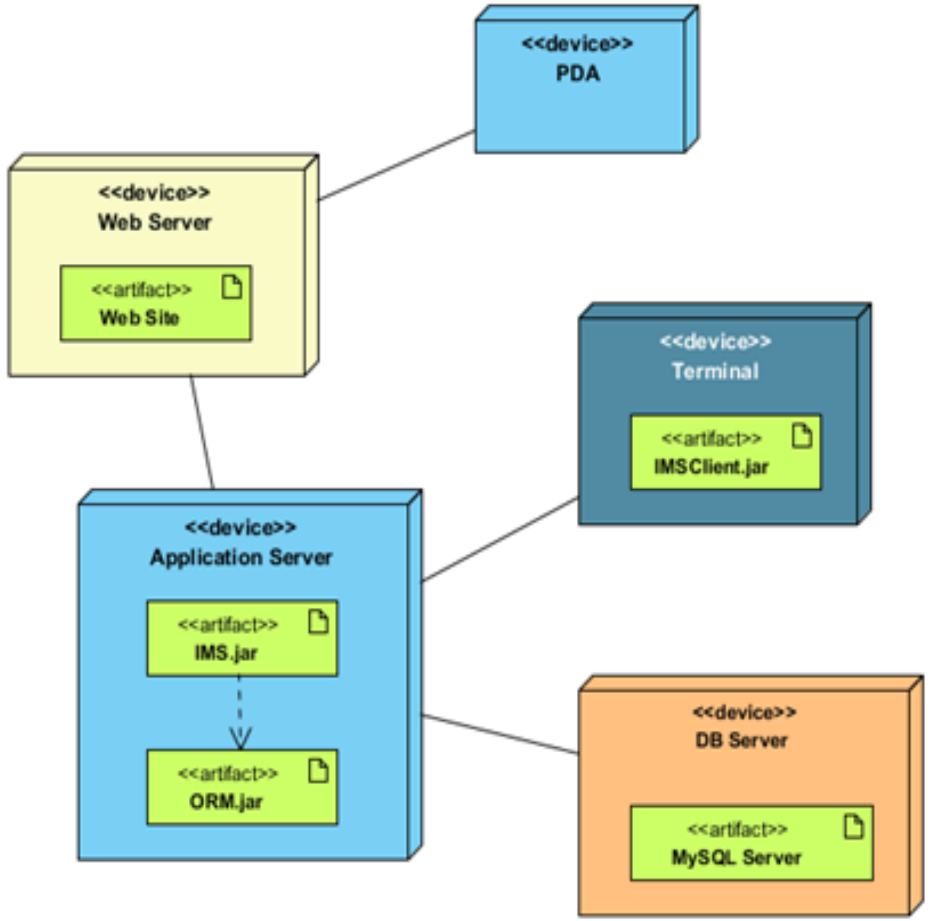
Ví dụ Sơ đồ Triển khai (Đơn giản)

- Sơ đồ Triển khai cho kiến trúc client-server 2 tầng.
- ví dụ hệ thống đặt lịch hẹn)
- ví dụ:
  - Nodes:
    - Receptionist PC (Client Workstation) và Office Server (Server).
  - Communication Path:
    - Kết nối giữa 2 node là <<LAN>>.
  - Artifacts:
    - <<HCI Layer>> được triển khai trên Receptionist PC.
    - <<PD Layer>> và <<DM Layer>> được triển khai trên Office Server.



Ví dụ Sơ đồ Triển khai (Phức tạp - 3 Tầng)

- kiến trúc 3 tầng phức tạp hơn, thường thấy trong các ứng dụng web."
- ví dụ:
  - Nodes:
    - Client Browser, Web/App Server, Database Server.
  - Communication Path:
    - Internet/HTTP giữa Client và Web Server;
    - LAN/TCP-IP giữa Web Server và Database Server.
  - Artifacts:
    - Không có artifact cụ thể trên Client Browser (chỉ chạy HTML/JS).
    - WebApp.war (chứa Presentation & Business Logic) trên App Server.
    - CustomerDB (chứa Data) trên Database Server.



Lợi ích của Biểu đồ triển khai

- Biểu đồ triển khai cung cấp hình ảnh rõ ràng về cách các bộ phận phần mềm được đặt trên phần cứng
- Hỗ trợ thảo luận về cách thiết lập hệ thống, giúp việc thảo luận và quyết định các chiến lược triển khai trở nên dễ dàng hơn.
- Hỗ trợ lập kế hoạch và quản lý quy trình triển khai, đảm bảo tài nguyên được sử dụng hiệu quả và đáp ứng các yêu cầu của hệ thống.
- Tài liệu hữu ích để hiểu cách thiết lập hệ thống, hỗ trợ cho việc bảo trì và thay đổi trong tương lai.

Những thách thức của biểu đồ triển khai

- Sơ đồ triển khai trở nên phức tạp, đặc biệt là trong các hệ thống lớn với nhiều bộ phận.
- Việc tạo và hiểu sơ đồ triển khai cần có kiến thức về ký hiệu UML mà không phải ai cũng có.
- Việc cập nhật sơ đồ triển khai khi hệ thống thay đổi tốn nhiều thời gian và công sức.
- Biểu đồ triển khai có thể không hiển thị cách mọi thứ thay đổi theo thời gian hoặc trong quá trình sử dụng thực tế.
- Việc tạo sơ đồ triển khai thường cần rất nhiều người cùng làm việc, điều này có thể khá phức tạp.

Ba sơ đồ này không tồn tại độc lập.

- Chúng cung cấp các góc nhìn bổ sung cho nhau để tạo thành một bức tranh kiến trúc hoàn chỉnh."
  1. Package Diagram:
     - Tổ chức các lớp (classes) thành các nhóm logic.
  2. Component Diagram:
     - Cho thấy cách các lớp trong các gói được đóng gói thành các thành phần (components) vật lý, có thể triển khai.
  3. Deployment Diagram:
     - Cho thấy cách các thành phần (components) được triển khai và chạy trên các nút (nodes) phần cứng.
- (Sử dụng một hình ảnh minh họa luồng từ
  - Package -> Component -> Deployment)

Chuyển tiếp: Từ Kiến trúc đến Thiết kế Chi tiết

- "Kiến trúc hệ thống là nền tảng.
  - Sau khi đã có nền tảng vững chắc, chúng ta sẽ xây dựng các 'bức tường' và 'nội thất' cho ngôi nhà phần mềm của mình.
- Các bước thiết kế chi tiết."
  - Class and Method Design: Chi tiết hóa các lớp, xác định các phương thức, ràng buộc và hợp đồng.
  - Data Management Layer Design: Thiết kế cơ sở dữ liệu vật lý, ánh xạ lớp vào bảng.
  - Human-Computer Interaction Layer Design: Thiết kế chi tiết giao diện người dùng, luồng điều hướng.
  - Các mô hình kiến trúc đóng vai trò kim chỉ nam cho tất cả các hoạt động này.

Tổng kết

- Kiến trúc hệ thống là cầu nối giữa
  - Phân tích ("cái gì") và Thiết kế ("làm thế nào"),
  - thiết lập một tầm nhìn tổng thể cho hệ thống.
- Kiến trúc phần mềm /Kiến trúc Kỹ thuật (Vật lý)
  - Các sơ đồ kiến trúc UML
    - Class/Object Diagrams
    - Package Diagram.
    - Component Diagram,
    - Deployment Diagram,
- Các chiến lược thiết kế cần được đánh giá cẩn thận dựa trên nhiều yếu tố của dự án và tổ chức.
  - (Tùy chỉnh, Mua gói, Thuê ngoài)

HUST

hust.edu.vn fb.com/dhbkhn

Bài tập Tình huống: Kiến trúc Hệ thống Đặt vé Xem phim Online

- Mô tả Bài toán
  - "Để củng cố kiến thức, hãy cùng áp dụng những gì đã học vào một ví dụ thực tế.
  - Hãy phác thảo kiến trúc cho một hệ thống đặt vé xem phim trực tuyến."
- Mô tả:
  - Người dùng xem lịch chiếu, chọn suất, chọn ghế, thanh toán. Hệ thống giao tiếp với cổng thanh toán bên ngoài. Triển khai trên nền tảng web.
- Yêu cầu:
  - Đề xuất kiến trúc vật lý, vẽ Sơ đồ Triển khai, Sơ đồ Thành phần và Sơ đồ Gói.

Lời giải - Sơ đồ Thành phần

- Sơ đồ Thành phần:
- Components: MovieUI.war, BookingEngine.jar, PaymentAdapter.jar, DatabaseConnector.jar.
  - Interfaces:
    - BookingEngine.jar cung cấp IBookingService.
    - PaymentAdapter.jar yêu cầu IPaymentGatewayService.
    - DatabaseConnector.jar cung cấp IDataAccess.
  - Dependencies:
    - MovieUI phụ thuộc BookingEngine,
    - BookingEngine phụ thuộc PaymentAdapter và DatabaseConnector.

Lời giải - Sơ đồ Triển khai và Gói

- Kiến trúc vật lý: 3 tầng (3-tier) là phù hợp.
- Sơ đồ Triển khai:
  - Nodes: User Browser, Web/App Server, Database Server, Payment Gateway Server (external).
  - Artifacts: BookingWebApp.war trên App Server, MovieDB trên DB Server.
- Sơ đồ Gói (Tầng kiến trúc):
  - Packages: UI Layer, Business Logic Layer, Data Access Layer.
  - Dependencies: UI -> Business Logic -> Data Access.

Thảo luận về Lựa chọn Thiết kế

- "Với hệ thống đặt vé này,
  - chúng ta nên tự xây dựng,
  - mua gói có sẵn
  - hay thuê ngoài?
- Hãy cùng phân tích."
  - Tùy chỉnh:
    - Phù hợp nếu muốn tạo trải nghiệm người dùng độc đáo và tích hợp sâu với các hệ thống khác của rạp chiếu phim.
  - Mua gói:
    - Có thể có các gói phần mềm quản lý rạp chiếu phim đã bao gồm chức năng đặt vé online. Cần đánh giá xem có đáp ứng đủ nhu cầu không.
  - Thuê ngoài:
    - Phù hợp nếu không có đội ngũ kỹ thuật mạnh hoặc muốn tập trung vào kinh doanh chính là vận hành rạp.